

Steerable Observability for BSP Systems

Ankush Jain

Thesis Defense, March 20th, 2026

Committee:

Prof. George Amvrosiadis (Chair)

Prof. Greg Ganger

Dr. Brad Settlemyer

Prof. Vyas Sekar

Prof. Andy Pavlo



Carnegie Mellon University

Parallel Data Laboratory

This Talk: *Efficiency via Feedback Loops*

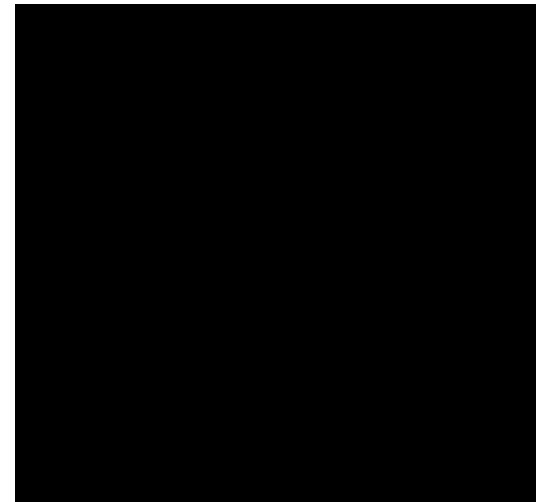
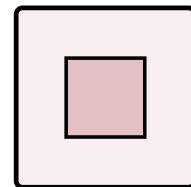
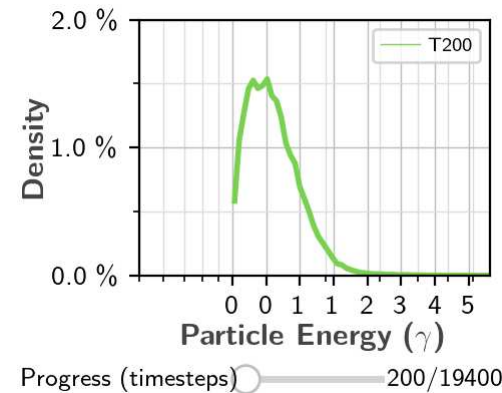
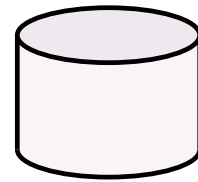
Setting

Supercomputers

- **BSP**: ML, science
 - *Weather/climate*
 - *Aerodynamics*
 - *Plasma/fusion*
- 10,000s of nodes
- Expensive and fragile

Challenges

Data & Compute Efficiency



Solution

General Feedback Loops

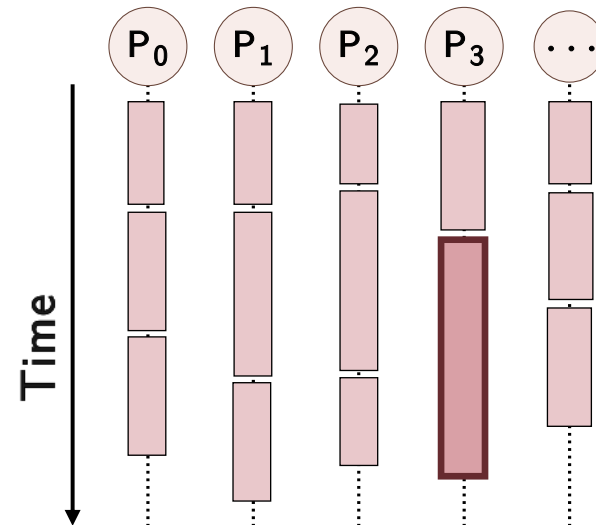
*Major efficiency bottleneck:
unpredictable conditions,
solutions simple/idiosyncractic*

Real bottleneck:
*characterizing and
reacting quickly*

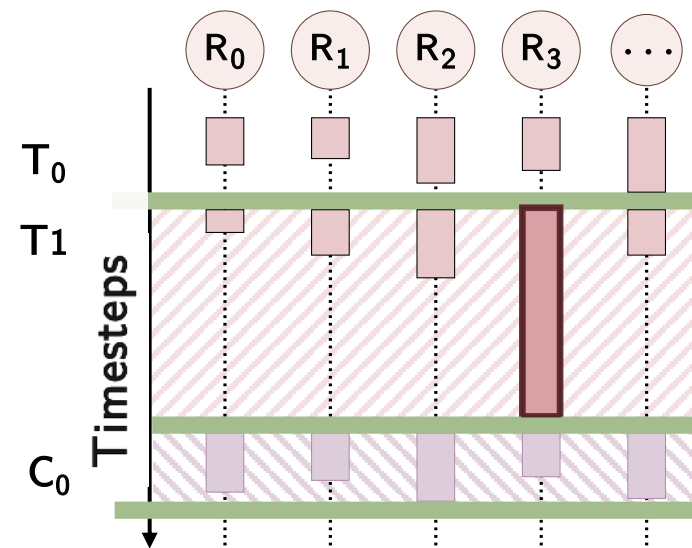
Steerable Observability:
See fast, act fast

BSP: *Synchronization Changes The Rules*

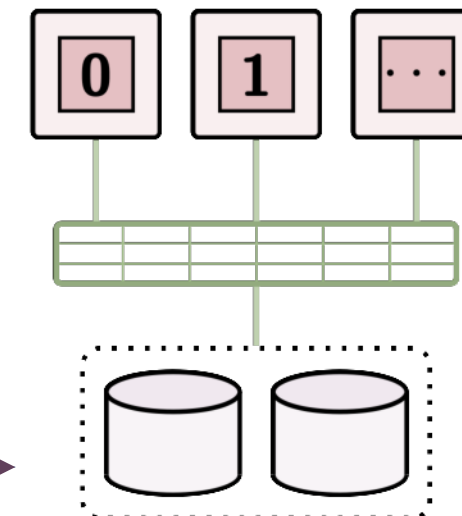
Cloud: *asynchronous*
(mostly)



BSP: *Bulk-Synchronous*
Parallel



... needs a “supercomputer”



BSP Ranks
10–100K

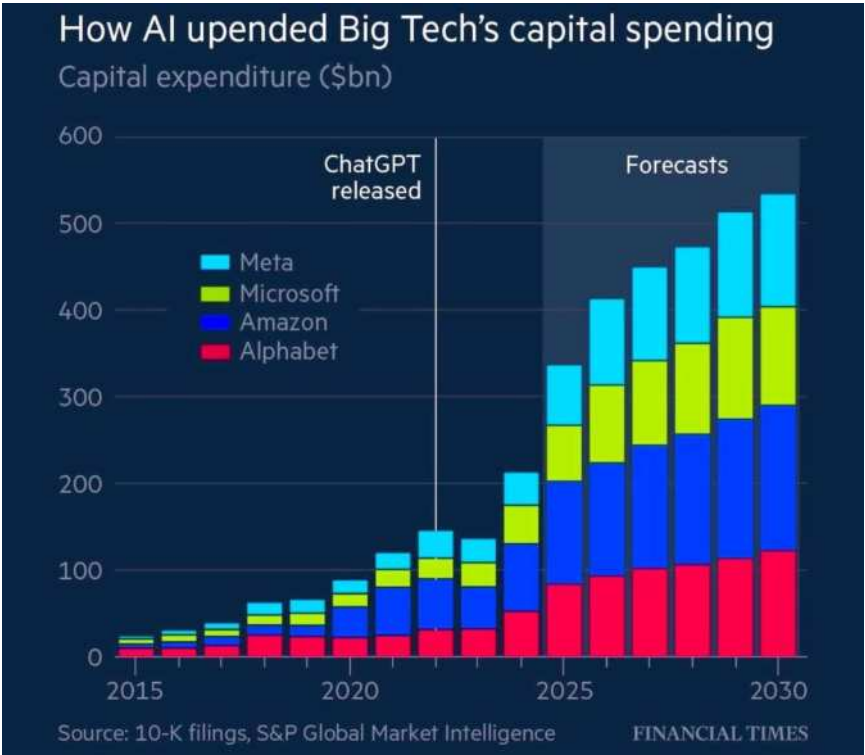
RDMA Fabric
400 Gbps

Parallel Storage
(Lustre, DAOS)

P Process **Work** **R** Rank **Global Sync. (MPI, NCCL)** **Checkpoint**

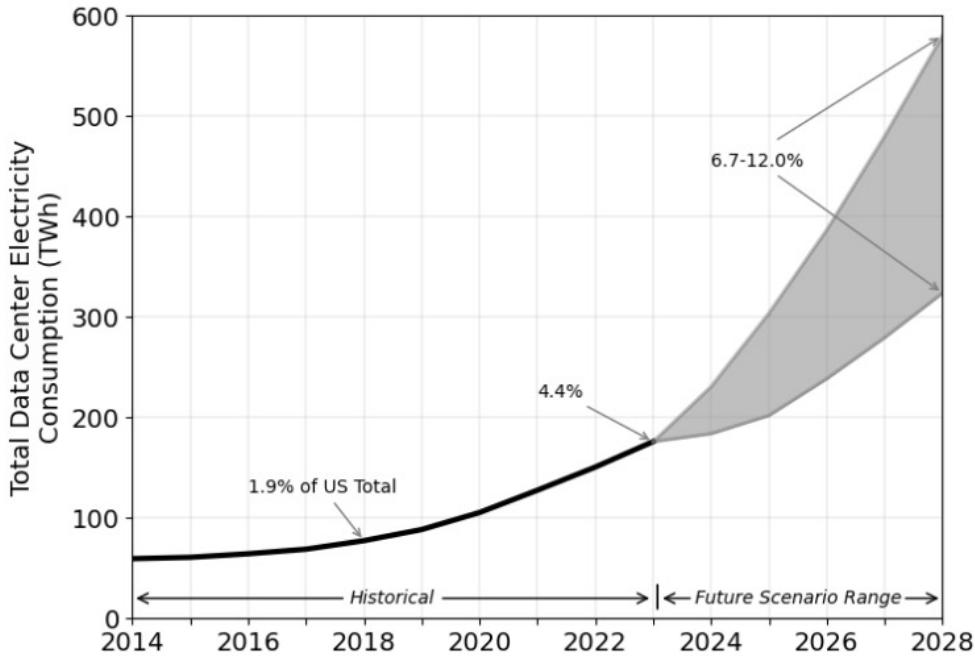
*“In large-scale training runs, the 100th percentile tail latency—not the median—dictates job completion. If even a single node or link lags, the entire training run can stall.
Our goal is to unlearn networking assumptions from the past...”*

What's At Stake? *Everything.*



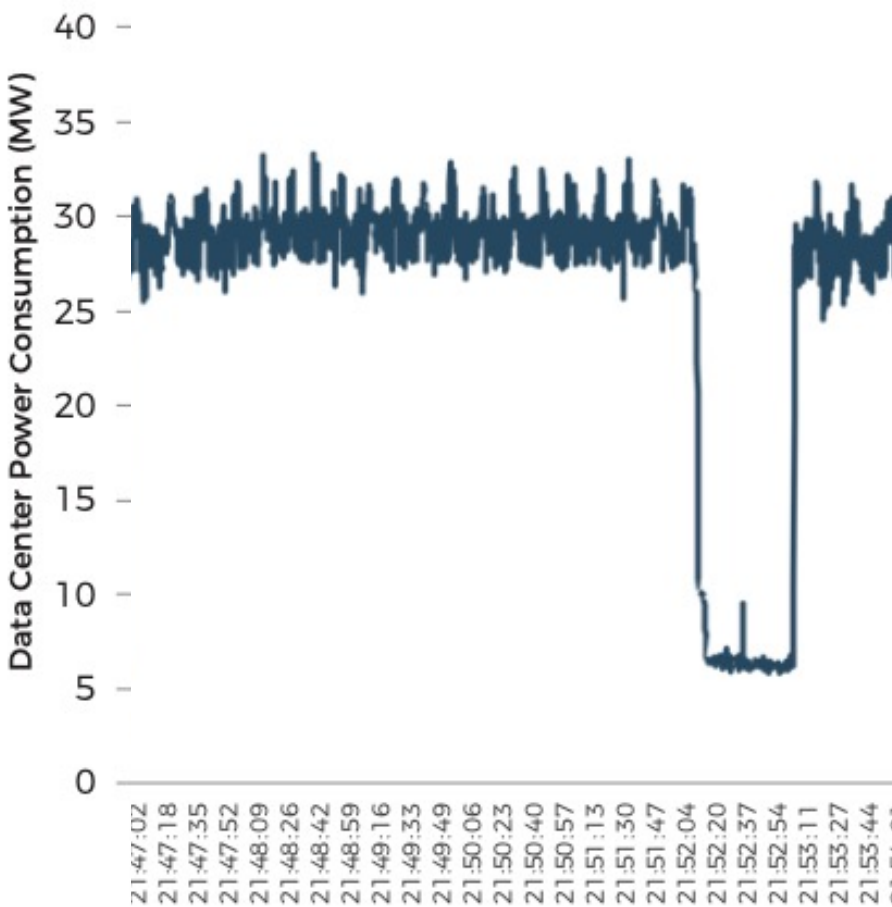
Tech capex: +\$300 B/year

Financial Times



**Datacenter electricity footprint
from 2% to 6.7-12% of total**

2024 United States Data Center Energy Usage Report, LBNL



thousands of GPUs, tensor processing units (TPU), and central processing units (CPU).⁴ These units often synchronize at certain steps, leading to periodic spikes and dips in power demand. Large-scale AI training involves massive and possibly cyclical or quasi-periodic surges in computational power demands. These effects may multiply if they are synchronized across data center locations.

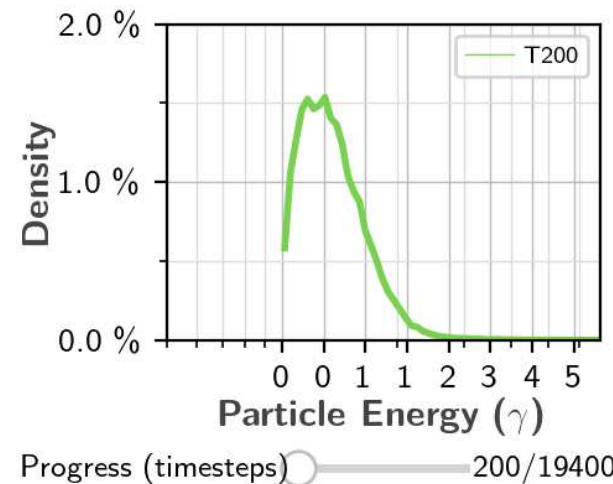
Sync affects grid stability!

Practical Guidance and Considerations for Large Load Interconnections, Elevate Energy Consulting, 2025

Our Initial Work: *Two Case Studies, One Motif*

How do we analyze data faster?

Can we organize data en route to storage?



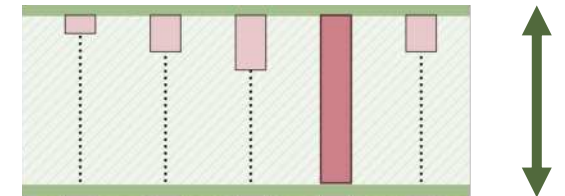
Challenge: data attributes highly skewed, and keep evolving

CARP: Streaming Data Reorganization

- **Online feedback loop, effective, works**
- Sort-like range query perf, ingests 5X faster



How do we compute efficiently?



AMR: dynamic mesh codes

Challenge: load imbalance?

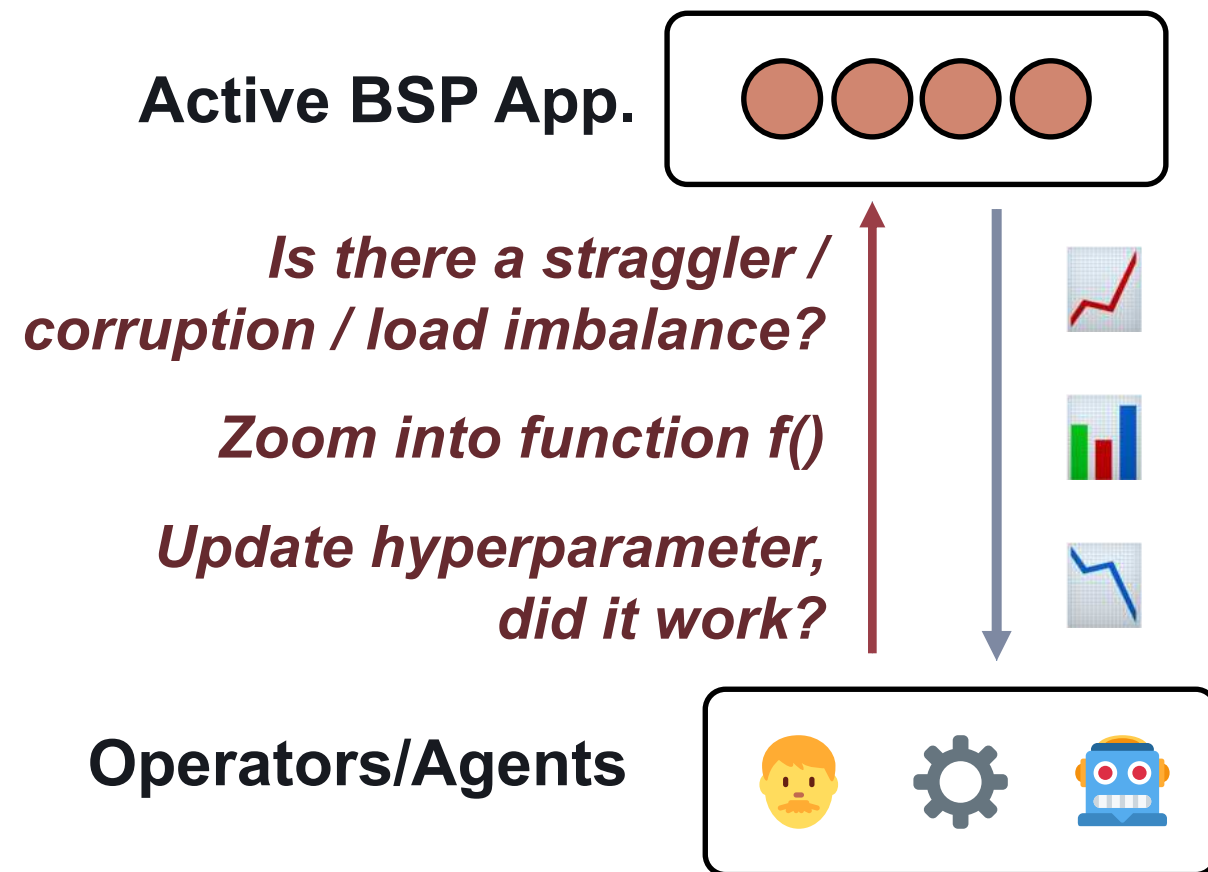
Lessons from AMR Placement

- Better placement → 21.6% faster codes
- Many other culprits: *bad hardware, tuning*
 - Each took months to diagnose
- **The real bottleneck is feedback**
 - Reason: tracing + offline analyses

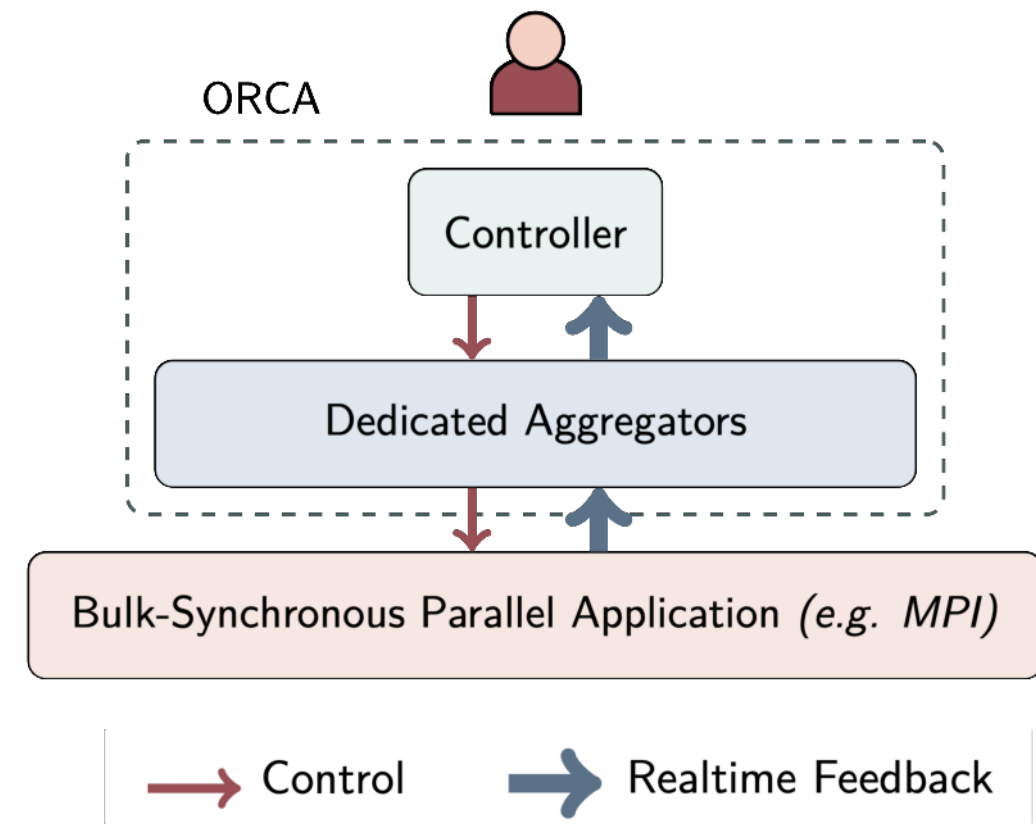
Lesson: *Build Programmable Feedback Loops!*

Steerable Observability: *always-on, programmable feedback and control*

- **Feedback:** “*useful views*” materialized directly, **control:** interventions



ORCA: Observability with Real-time Control and Aggregation

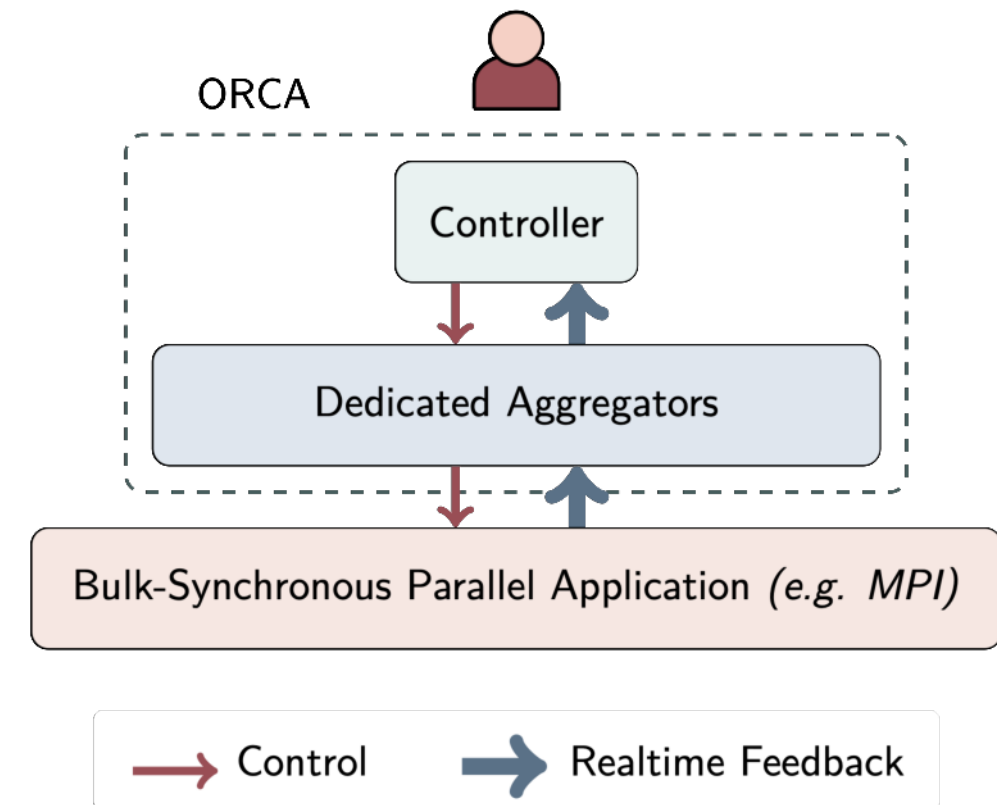


Thesis Statement

A large class of efficiency bottlenecks in BSP systems arise from conditions unknowable before runtime.

Such conditions are systematically addressable through adaptive mechanisms if the requisite feedback loops exist, and perpetuate if they do not.

A general-purpose programmable substrate can close these loops at scale.



Talk Outline

- **Data path:** *CARP (Continuous Adaptive Range Partitioner)*
- **Compute path:** *Lessons from AMR Placement*
- **ORCA** (*Observability with Real-time Control and Aggregation*)
- **Conclusions and Future Directions**

CARP: Single-Pass Range Query Indexing

Problem: *ingesting checkpoint data for range queries*

Challenges

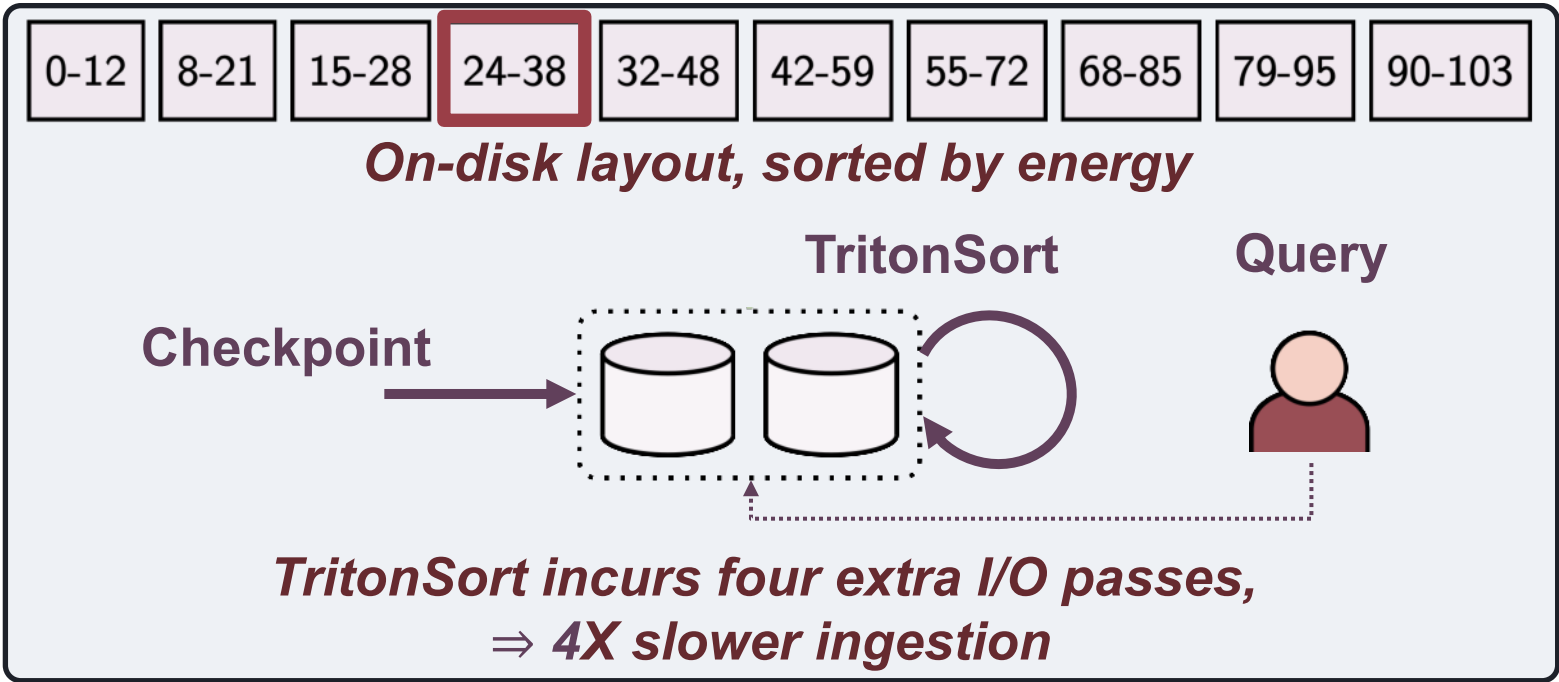
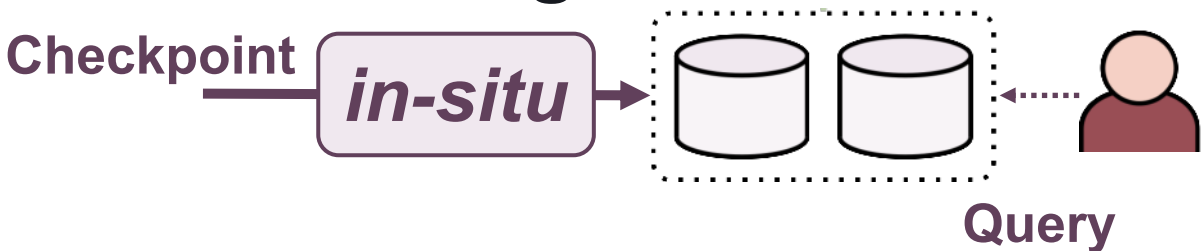
- Checkpoints are synchronized I/O bursts
- Storage bandwidth bottleneck
- Reorganization reduces ingestion bandwidth

Particle	Energy	X	Y	Z
P ₀	52.3	0.23	0.87	0.45
P ₁	23.7	0.56	0.12	0.78
P ₂	89.2	0.91	0.34	0.22
P ₃	22.1	0.18	0.65	0.91
P ₄	12.4	0.72	0.41	0.55

Range query:
energy between [22, 24)

Sorting: *query-efficient but expensive to produce*

Can we reorganize in-situ?



Challenge: *Unknown Key Distributions*

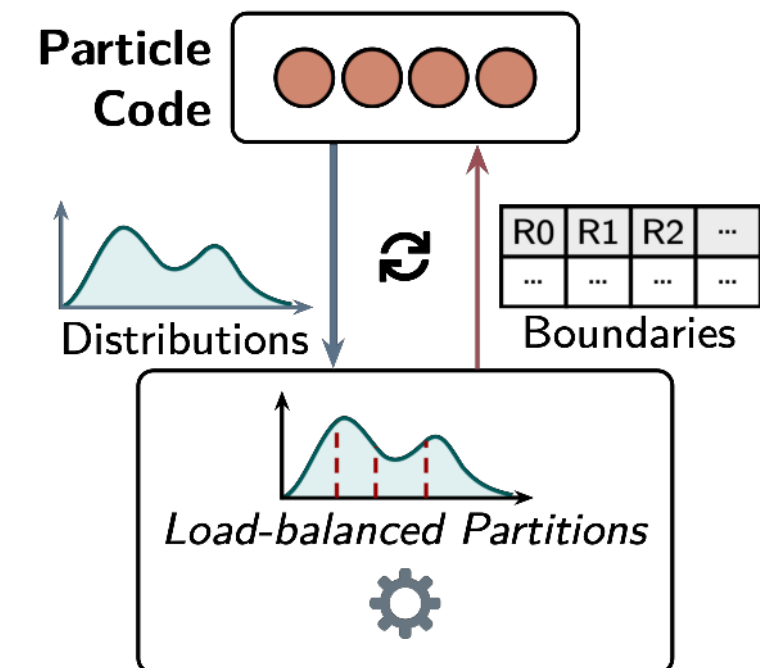
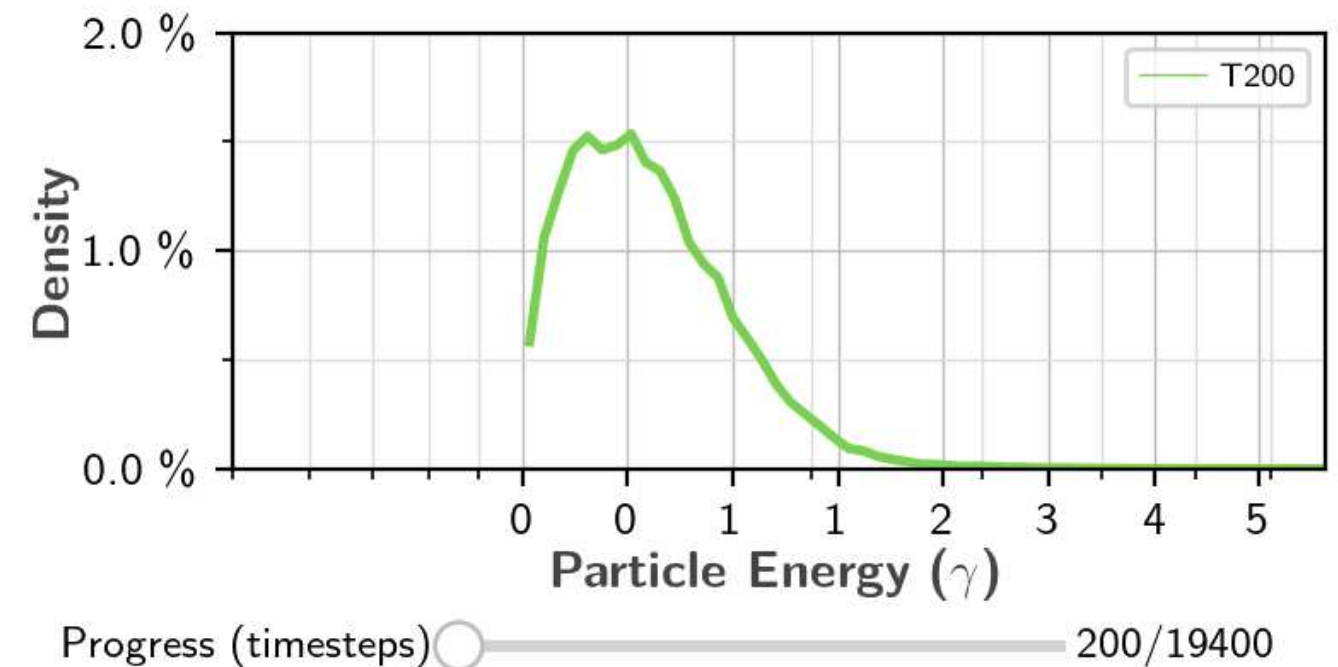
Can we “sort” data as it streams?

Challenges

- *Scientific attributes are highly skewed, dynamic, and evolve over time*
- *No one knows the global distribution (every rank sees a partial view)*

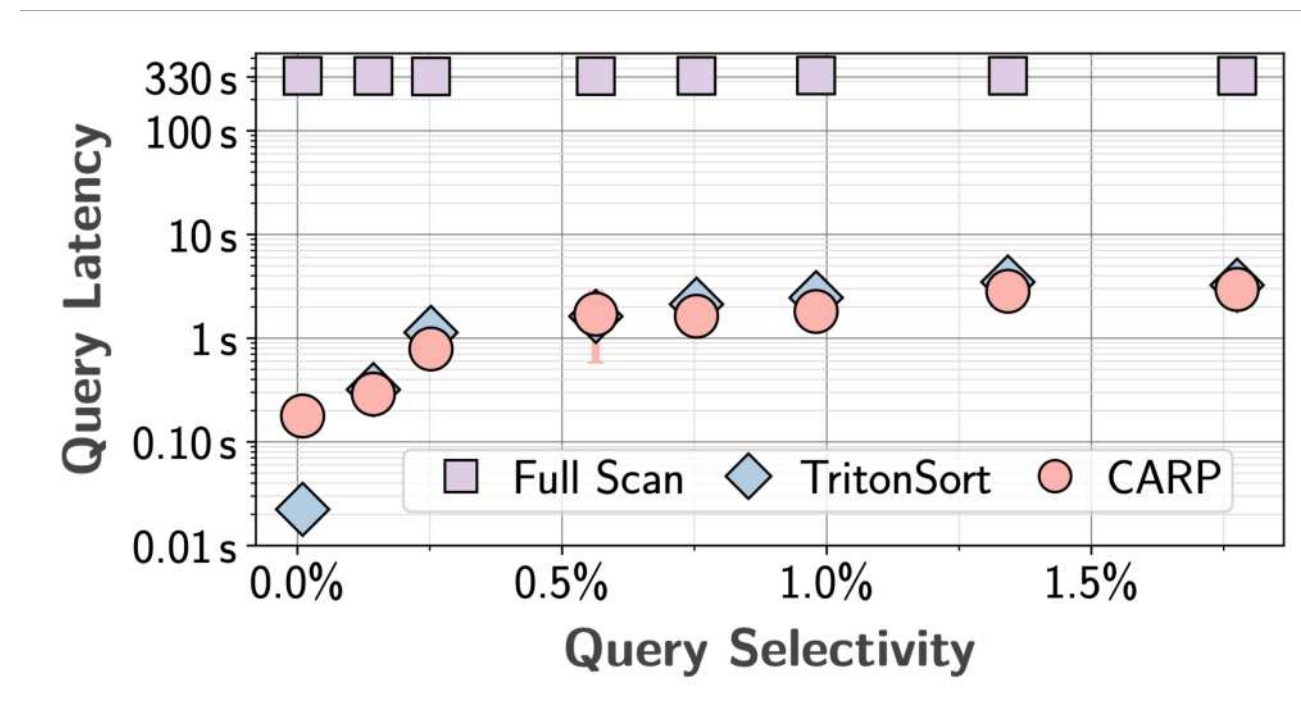
CARP: *An Adaptive, Greedy Approach*

- Discover current global distribution at runtime
- Use global view to partition incoming data
- Never move previously written data



Eval: *No Ingestion Overhead, ~Optimal Queries*

- **Ingestion:** no overhead, as fast as unmodified checkpoint
 - **4.9X faster** than TritonSort (*distributed out-of-core sort*)
- **Query performance:** *almost as good as a full sort!*
- **Lesson:** *adaptive loops work, if they exist*



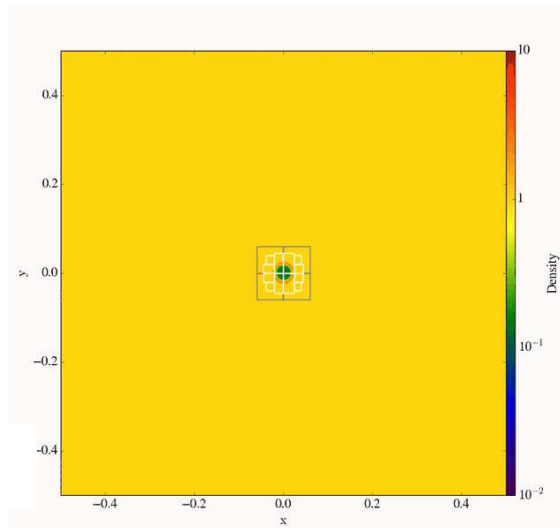
Talk Outline

- **Data path:** *CARP (Continuous Adaptive Range Partitioner)*
- **Compute path:** *Lessons from AMR Placement*
- **ORCA** (*Observability with Real-time Control and Aggregation*)
- **Conclusions and Future Directions**

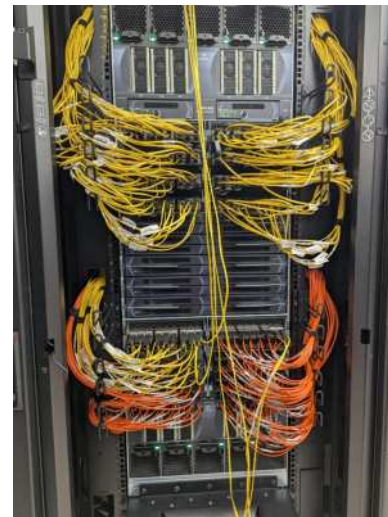
Lessons From Optimizing AMR Placement

- **AMR** (*Adaptive Mesh Refinement*): dynamic mesh codes
 - Load imbalances widely acknowledged but uncharacterized
- **Our goal**: placement using runtime telemetry
 - Challenges: *execution variability, tuning issues, jitter etc.*
- **Contributions**: placement policy (CPLX) + lessons
 - Lesson: *real problem is knowing your problem*

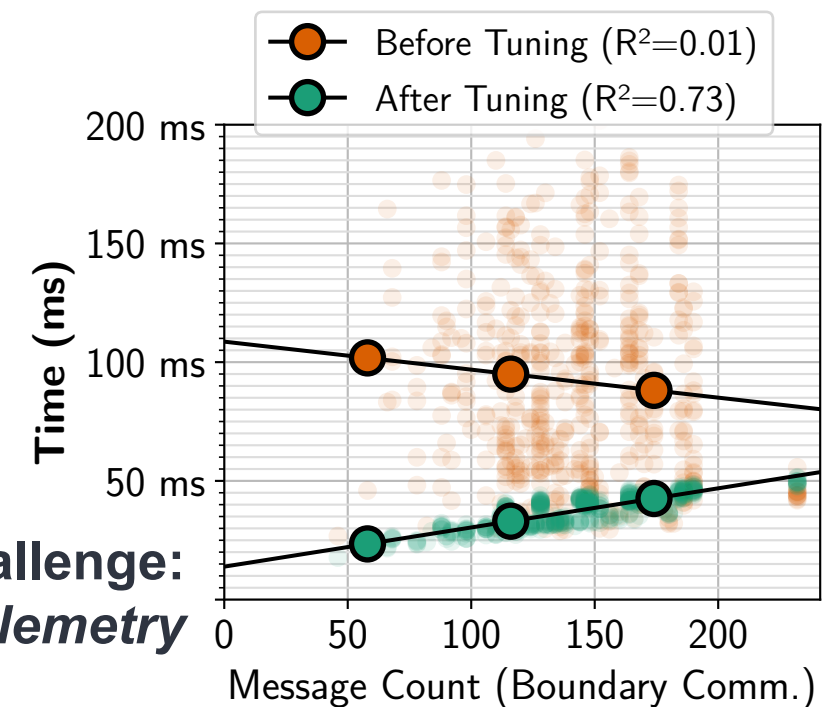
AMR Code



Wolf Cluster
@ CMU PDL



Challenge:
Noisy Telemetry



Key Challenge: *Visibility Into Runtime Behavior*

AMR has complex and fine-grained interactions. What matters?

Profiling: *coarse-grained*
Does not identify root causes

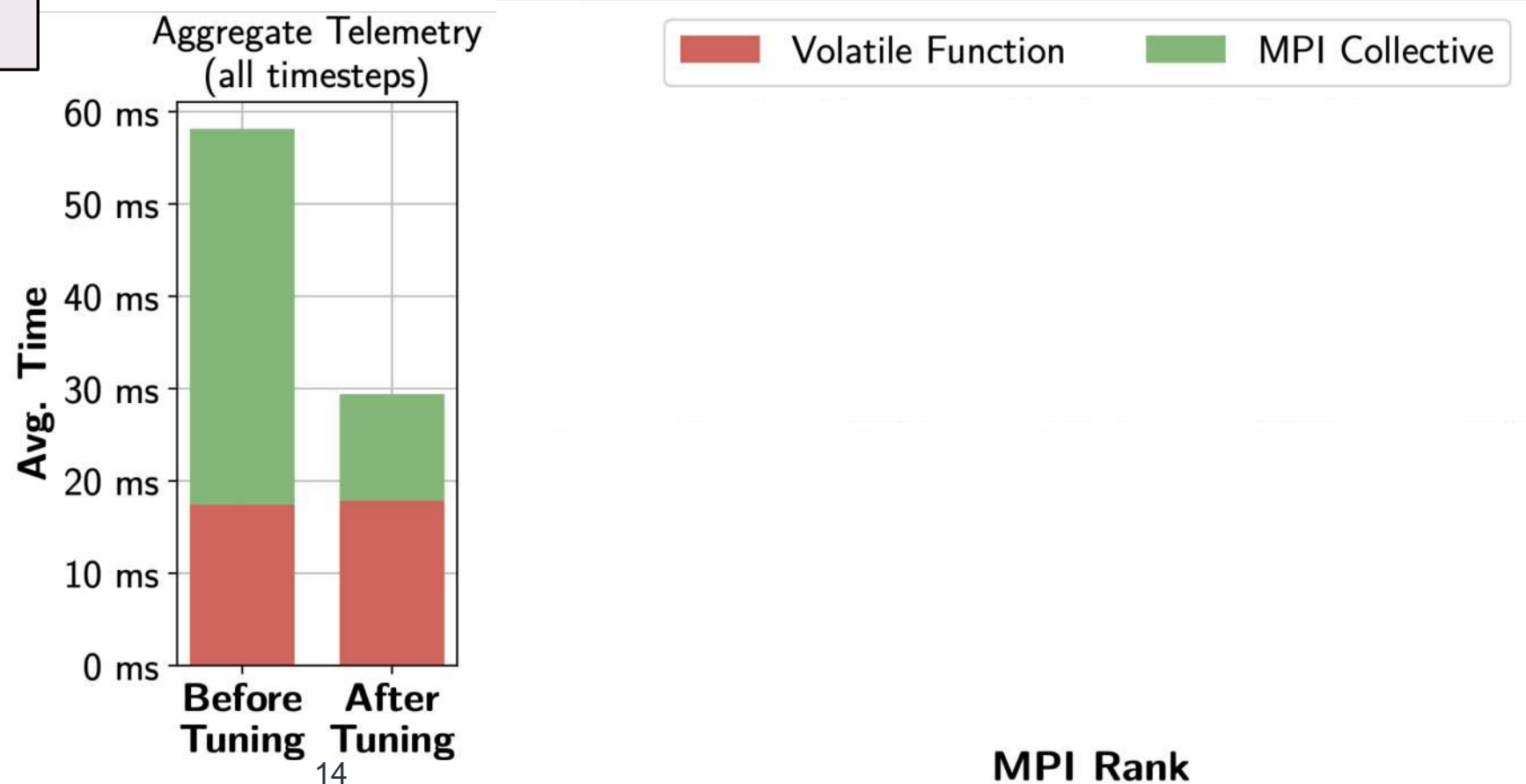
<i>Physics()</i>	12%
<i>Task_Wait()</i>	45%
<i>MPI_Barrier()</i>	31%

Event tracing: *fine-grained*

- "Every" function + duration written as JSON
- $O(100 \text{ GB/timestep})$, $O(50K)$ timesteps

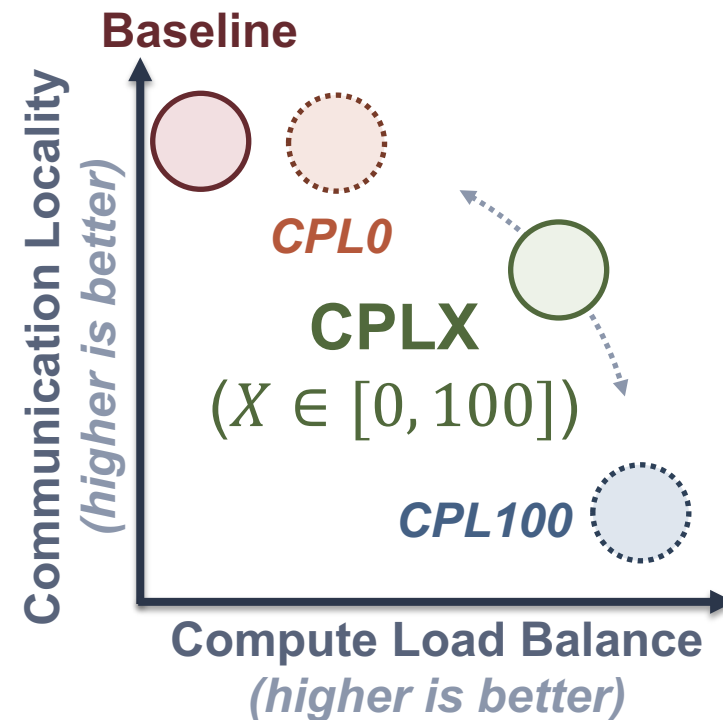
Kept switching tools:
profiling, tracing, perf,
custom hooks, eBPF...

Diagnoses would take months, fixes took days



Placement: *NP-Hard, Tradeoffs, $O(ms)$ Budget*

- Placement: *tradeoff between load balance and locality*
- **CPLX**: tunable control, *practically* pareto-optimal



Results @ 4096 ranks:

- Worst CPLX tune: **-17%**, best: **-22%**
- How important is locality? Unclear
- **Takeaway:** *tunables encode fundamental tradeoffs*

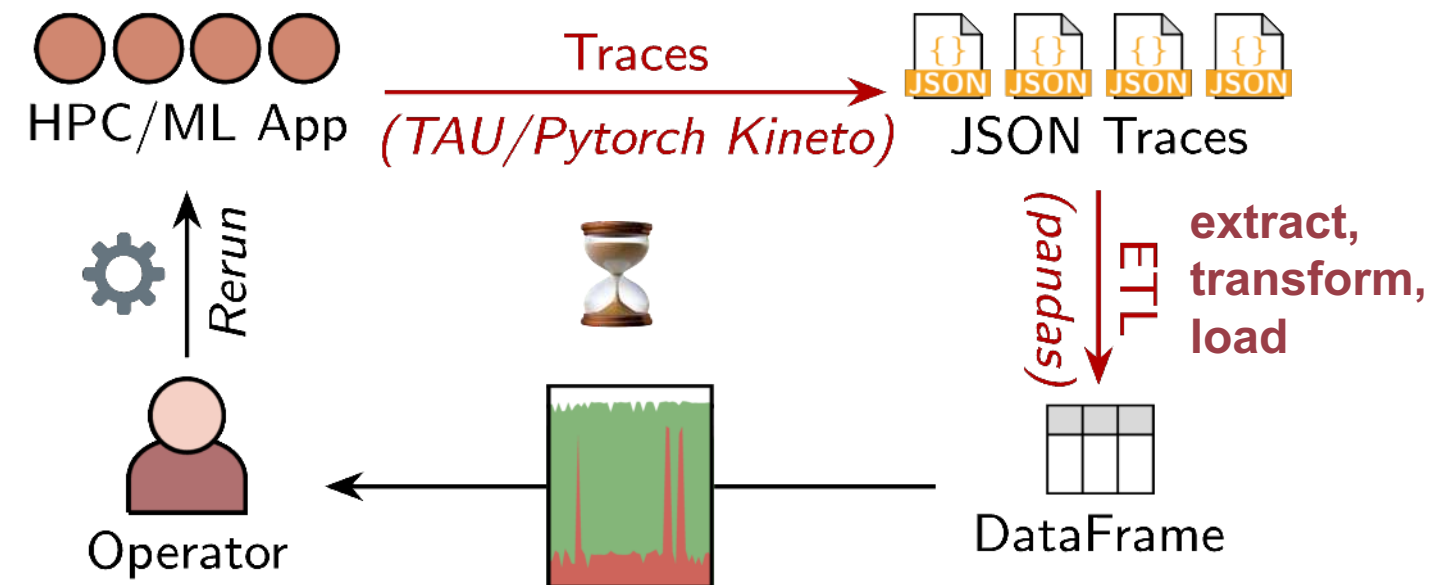
Lesson: AMR Is A Textbook Case of Unknowns



Is the problem load imbalance?

- *Right setting for X?*
- *Effectiveness of placement heuristics?*

**Real problem: *hard to see
and test hypotheses***



profiling, tracing, perf



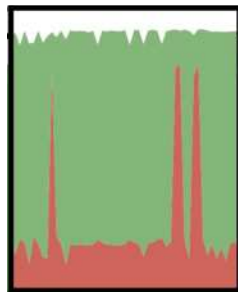
- All events are available, what do you need?
- *Limitation: node-scale, not cluster-scale*

Talk Outline

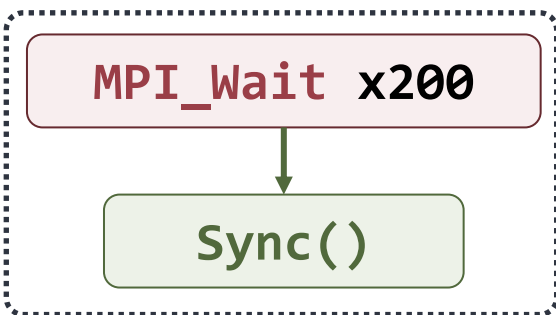
- **Data path:** *CARP (Continuous Adaptive Range Partitioner)*
- **Compute path:** *Lessons from AMR Placement*
- **ORCA** (*Observability with Real-time Control and Aggregation*)
 - **Overview and Prior Work**
 - Design
 - Evaluation
- **Conclusions and Future Directions**

What Does Better Observability Look Like?

Problem



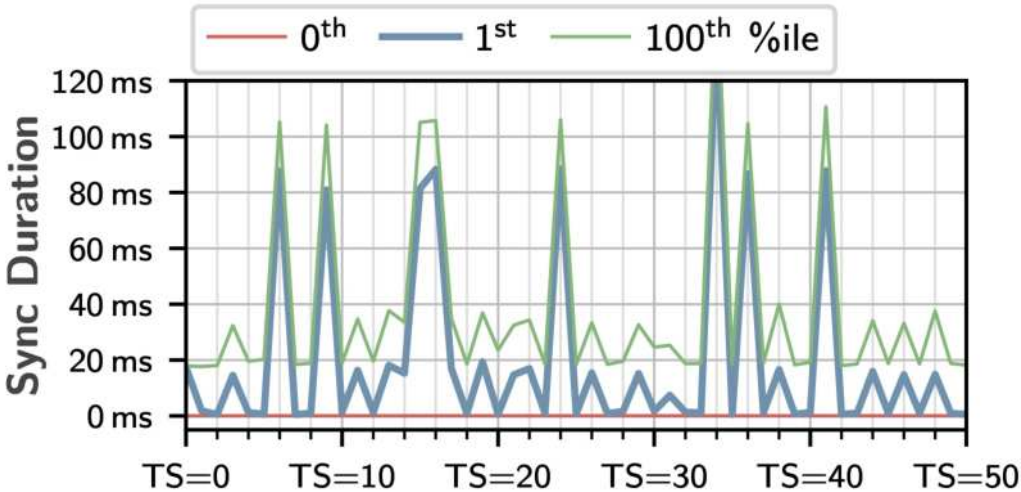
One rank, one timestep



x 7
steps/sec =

Trace event volume

4096 ranks	100K ranks
4M/sec	???
20K/sec	???
Other functions??	



What we have: a DoS attack

- **Cluster-scale event tracing**
- **Collection != insight**

What we need: feedback
Real-time, on-demand



Is there a problem?

Real-time sync() %-iles

~ 3 events/step

Is it MPI_Wait?

COUNT(MPI_Waits > 20ms)

~ 1 event/step

What sub-function?

...

What Does Better Observability Look Like?

Is there a problem?

Real-time sync() %-iles

~ 3 events/step

Is it MPI_Wait?

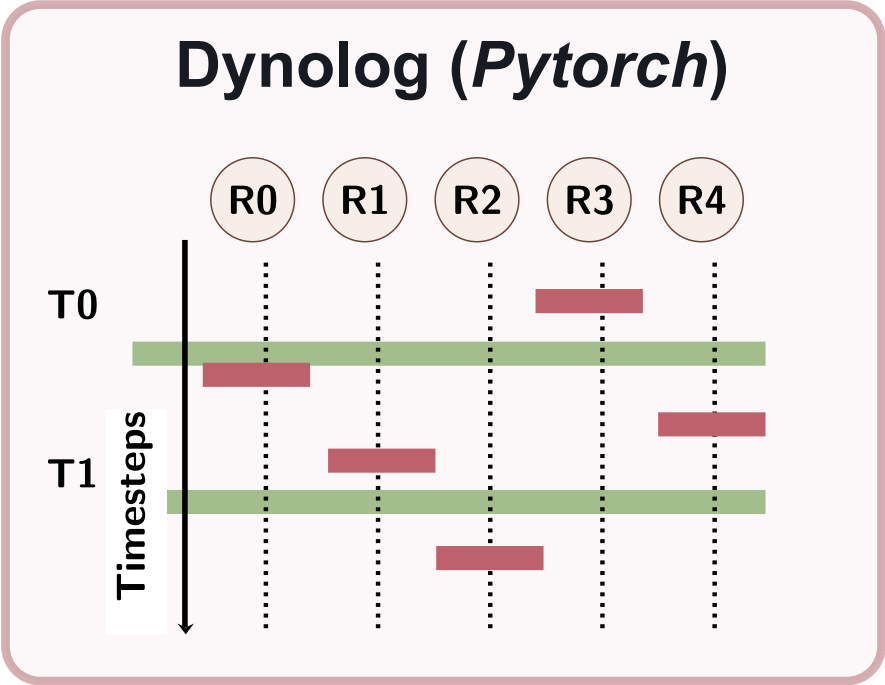
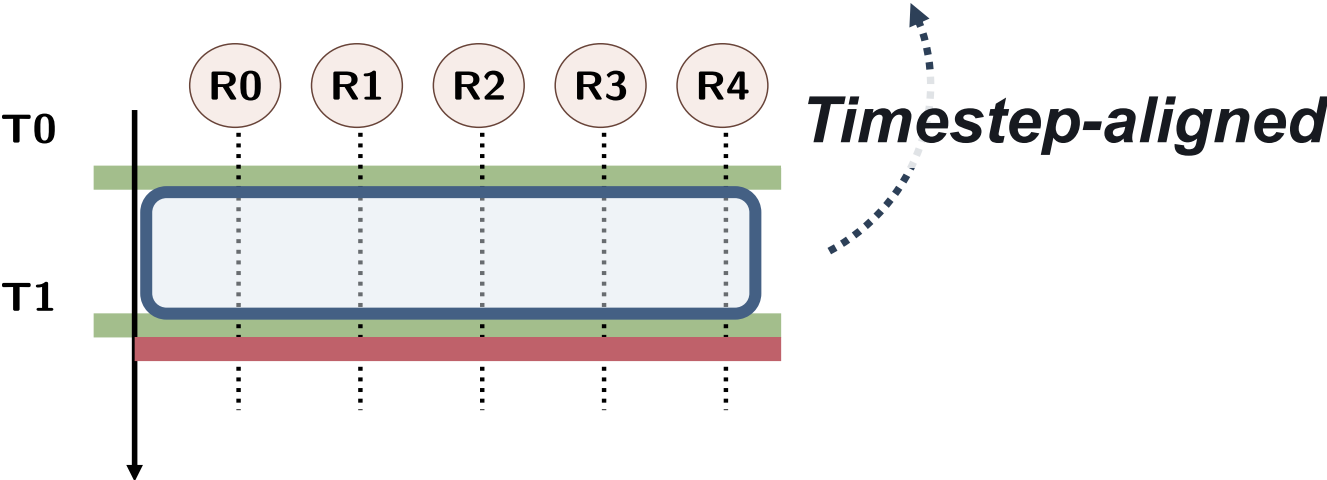
COUNT(MPI_Waits > 20ms)

~ 1 event/step

What sub-function?

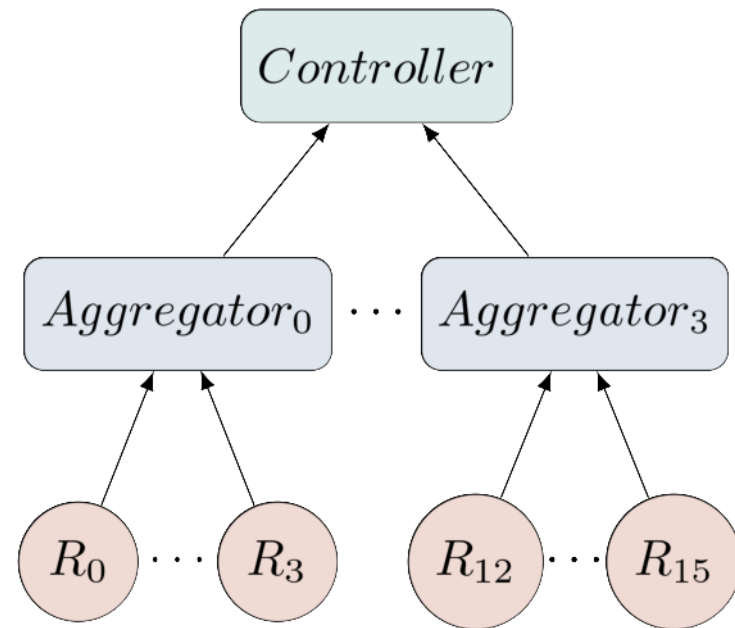
...

- **Feedback:** in-situ streaming aggregations
- **Control:** feedback-guided interventions



Steerable observability: steer application views, or the application itself

ORCA: Always-on *Steerable* Observability



On-Fabric Tree-based Overlay Network (TBON)

- *Controller: root, aggregators: scale-out*
- *Feedback: reductions ($\uparrow$), Control: broadcast ($\downarrow$)*
- *Analytics: SQL-based, pushdowns, columnar*



Compared to tracers:

- *44X smaller* traces
- *7800X faster* queries
- *< 2% overhead* (vs 56%-84%)

- 99.999% events discarded in-situ
- < 2% overhead

Claude + ORCA
debug  in 27 mins

Feedback Latency ($\rightarrow$ Is Better)

Offline

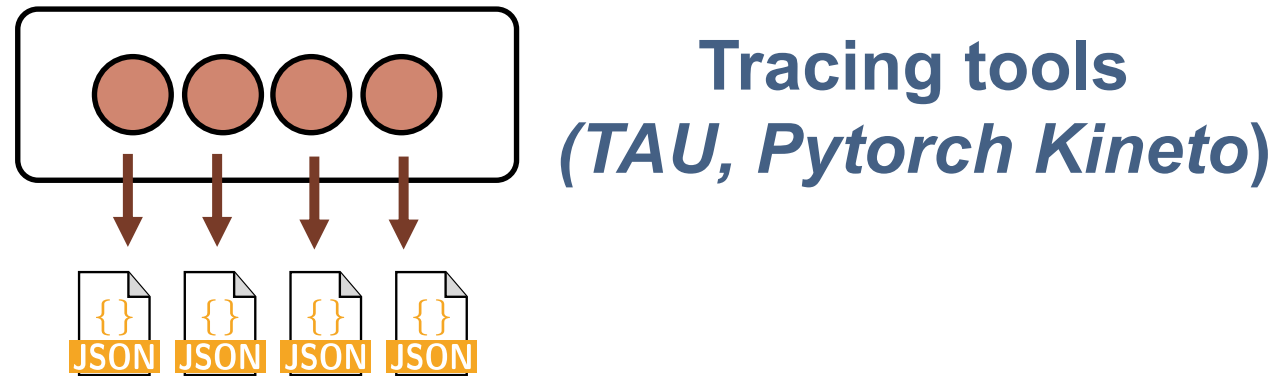
(traces + post-mortems)

Online

In-situ analytics

Steerable

HPC/ML Observability: *SoTA and Gaps*



Cons: overcollection, ETL

Can not:

- *be always-on*
- *terminate mid-way*
- *reconfigure mid-way*

Pro: programmability

- Ad-hoc analyses possible:
distributions, counts etc.

Online ML Observability Pipelines

Correctness

TrainCheck
OSDI '25

Performance Anomalies

Greyhound
ATC '25

Flare
NSDI '26

Cluster-level fault diagnosis

Holmes
NSDI '25

Mycroft
SOSP '25

Minder
NSDI '25

Aegis
NSDI '25

Reveal
arXiv '25

Pro: online

Cons: bespoke, limited scalability

Cloud Observability: *Practices and Gaps*

Traces (*distributed*)

Dapper, Jaeger etc.

- P99 in async. microservices
- ***BSP: steps, not requests***

Metrics

Prometheus etc.

- Known unknowns (*CPU util. %*)
- *No workload-level views*

Logs

- Per-process narratives
- *No workload-level views*

Talk Outline

- **Data path:** *CARP (Continuous Adaptive Range Partitioner)*
- **Compute path:** *Lessons from AMR Placement*
- **ORCA** (*Observability with Real-time Control and Aggregation*)
 - Overview and Prior Work
 - **Design**
 - Evaluation
- **Conclusions and Future Directions**

ORCA: Component Overview

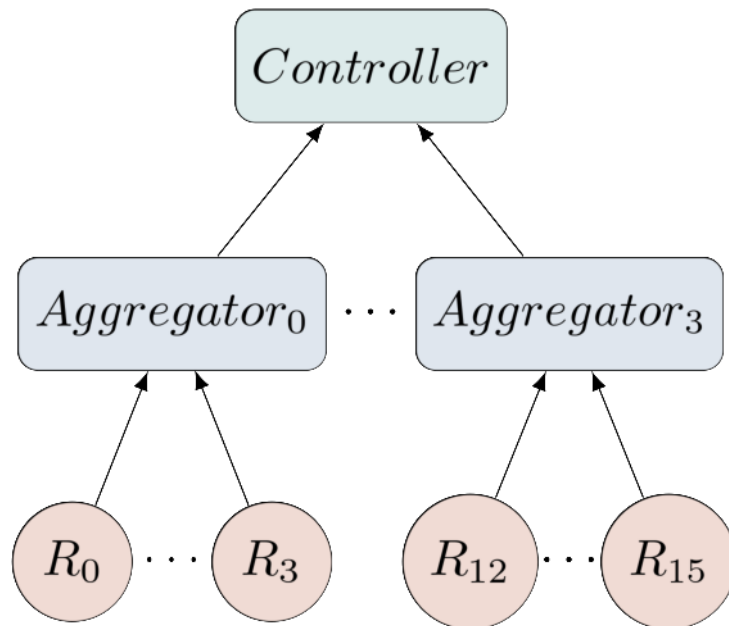
Transport: TBON, RDMA, incast-optimized

Data Model: *Timestep Dataframe*

Analytics: SQL-based, *columnar + pushdown*

Control: *async, timestep-consistent*

- Speculative two-phase commit, tied to a future timestep*



<i>Property</i>	Component
<i>BSP-awareness</i>	Co-design
<i>Scalability</i>	Tree-based overlay
<i>Low overhead</i>	Transport (RDMA), Arrow
<i>Usability</i>	SQL, workflow flexibility

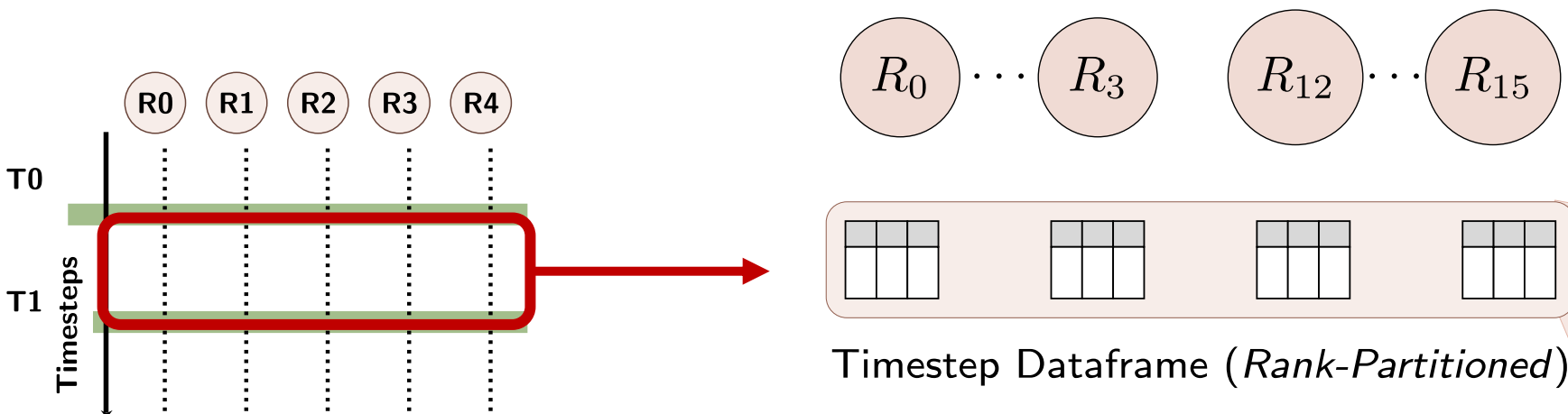
Data Model: *Streams & Dataframes*

Streams: event sources, have names + schemas

- Any source: *MPI, NCCL, CUPTI...*
- Drained every timestep into local Arrow dataframes



- *Column-major in-memory tables*
- *RDMA, zero-copy, ⚡ analytics*



***Timesteps are causally partitioned,
independent analysis units!***

🕒	Step	Rank	Function	Dura.
1513	671	0	MPI_Barrier	42 ms
1591	671	1	MPI_Barrier	67 ms
1409	671	2	MPI_Barrier	23 ms
			⋮	
1487	671	14	MPI_Barrier	89 ms
1552	671	15	MPI_Barrier	51 ms

Per-Stream Timestep Dataframe
Rank-partitioned (*like Spark RDDs*)

Analytics: SQL, Pushdowns, Sinks

OrcaFlow

SQL transforms + sinks

```
SELECT * FROM MPI_Waits
WHERE dura > 10 ms
```

Write to sink
(DuckDB, Parquet)

TierPlan
• Filter

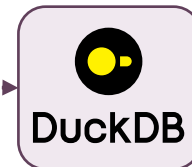
Ops & sinks placed automatically



Carnegie Mellon University

Parallel Data Laboratory

Sinks



Derived metrics

ts0_4/
r0_4 r4_8 r8_12 r12_16

ts4_8/
r0_4 r4_8 r8_12 r12_16

Timestep and rank-partitioned Parquet

Offline traces

Tier CTL

Controller

Tier AGG

Aggregator₀ ...

Aggregator₃

Tier MPI

R₀ ... R₃

R₁₂ ... R₁₅



Timestep Dataframe (Rank-Partitioned)

Any combination of online + offline workflows!

Talk Outline

- **Data path:** *CARP (Continuous Adaptive Range Partitioner)*
- **Compute path:** *Lessons from AMR Placement*
- **ORCA** (*Observability with Real-time Control and Aggregation*)
 - Overview and Prior Work
 - Design
 - ***Evaluation***
- **Conclusions and Future Directions**

Evaluation: Setup

Workload: *same AMR code, 512-4096 CPU ranks*

- 16 ranks/node (*4096 ranks = 256 nodes*)
- *CPU simulation $\Rightarrow$ strenuous test. **All cycles count!***

1 aggregator

1024 ranks

ORCA Configuration

Compare with tracers:
runtime overhead, trace
size, query perf.

**Pushdown to
simulation:** costs
and benefits?

**Is the feedback
loop usable?**
Using Claude

Feedback Latency ($\rightarrow$ Is Better)

Offline

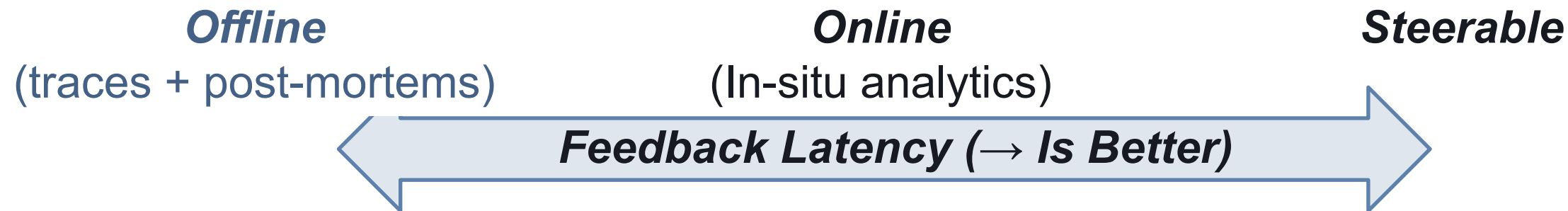
(traces + post-mortems)

Online

(In-situ analytics)

Steerable

Tracing: < 2% Overhead, 44X Smaller Traces



Runtime overhead: ~2% @ 4096 ranks
(vs **56—81%** for TAU and DFTracer)

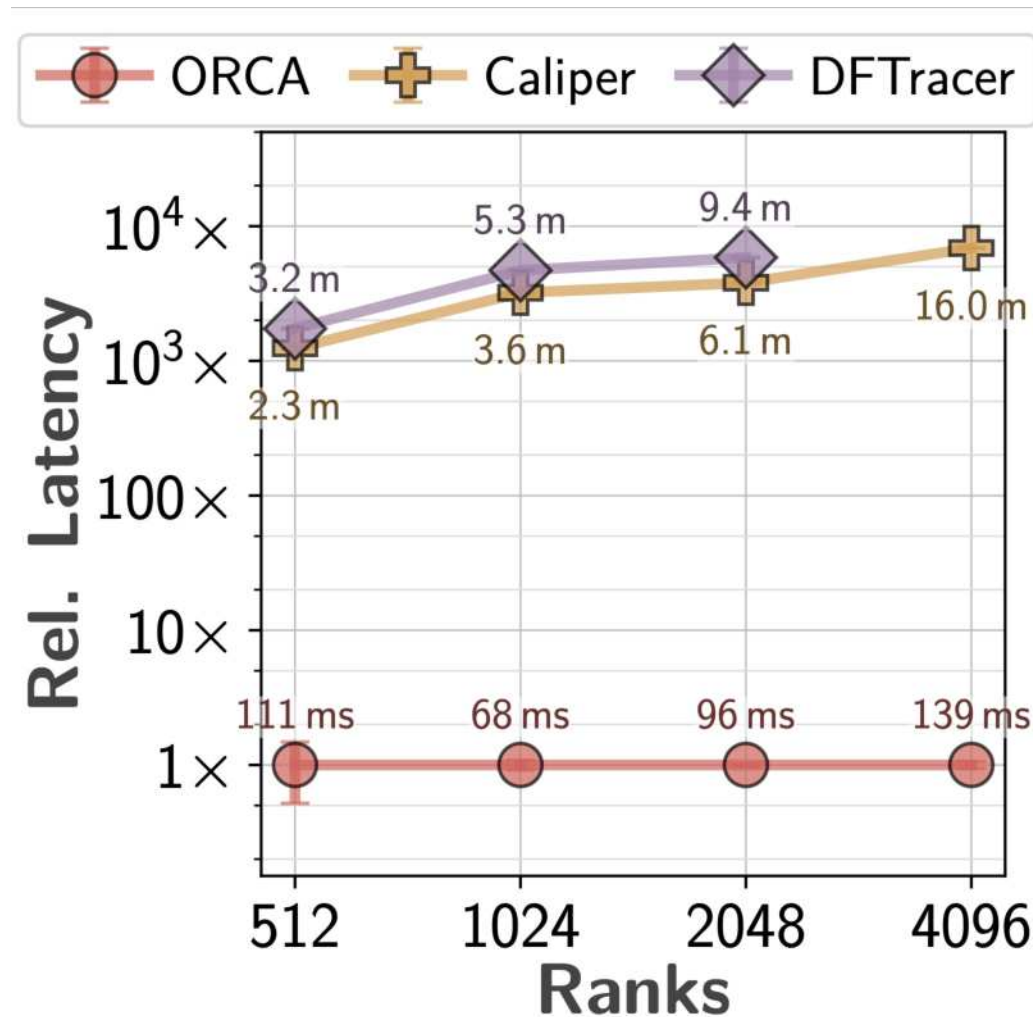
- *Tracers*: I/O-driven jitter
- *ORCA*: RDMA + pull-based model

Trace sizes: up to 44X Smaller
(equivalent collection profiles)

- *Tracers*:
 - Bloated formats
 - Hardcode derived views
- *ORCA*: parquet + compression

Offline Analytics: 7800X Faster Filtering *despite* compression

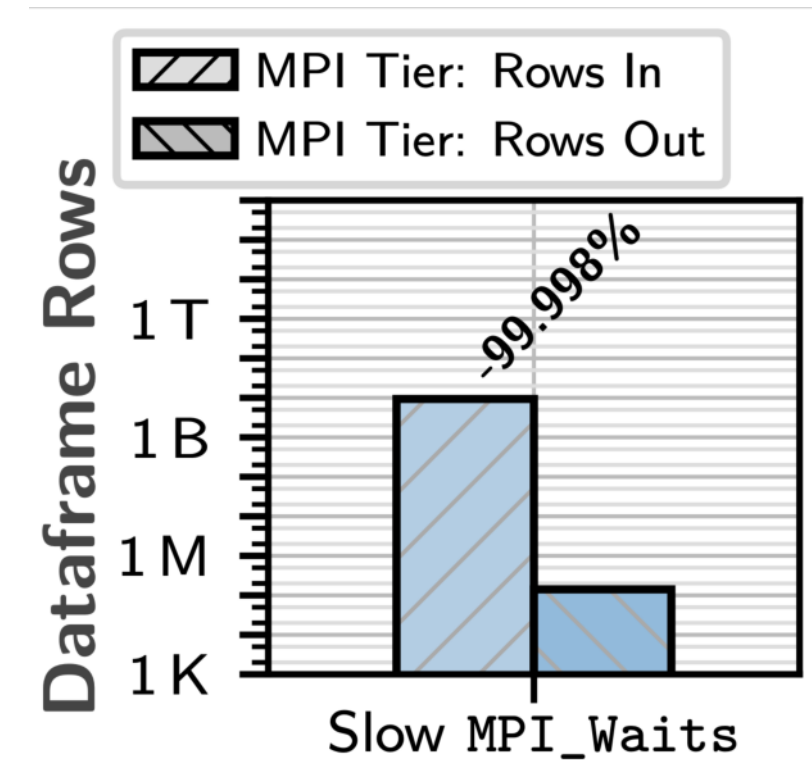
Query: MPI_Waits > 1ms



**TAU and Score-P excluded, as they do not generate queryable traces*

Parquet advantage: no ETL
(Partitioning not useful for this query)

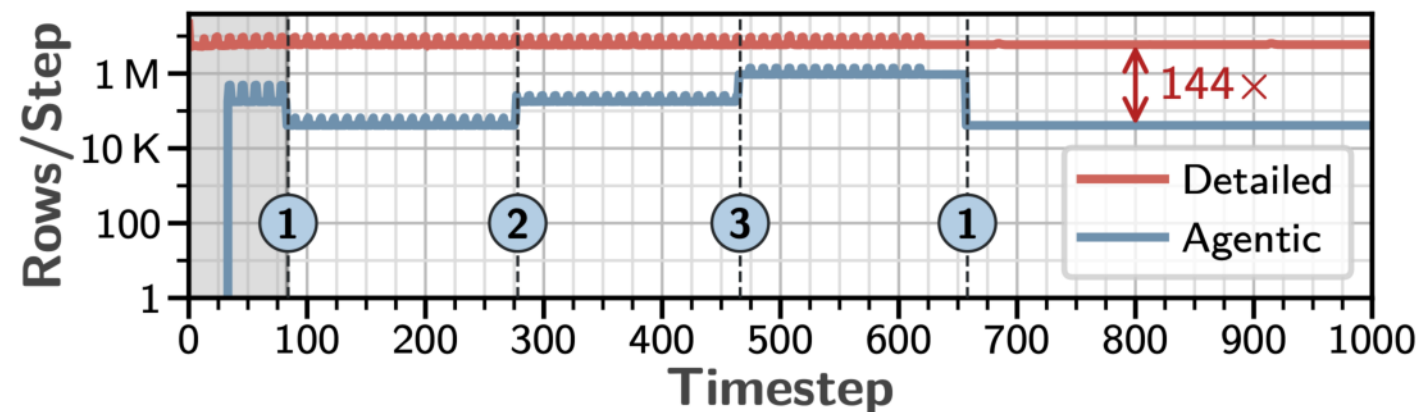
But ORCA does not need to persist!



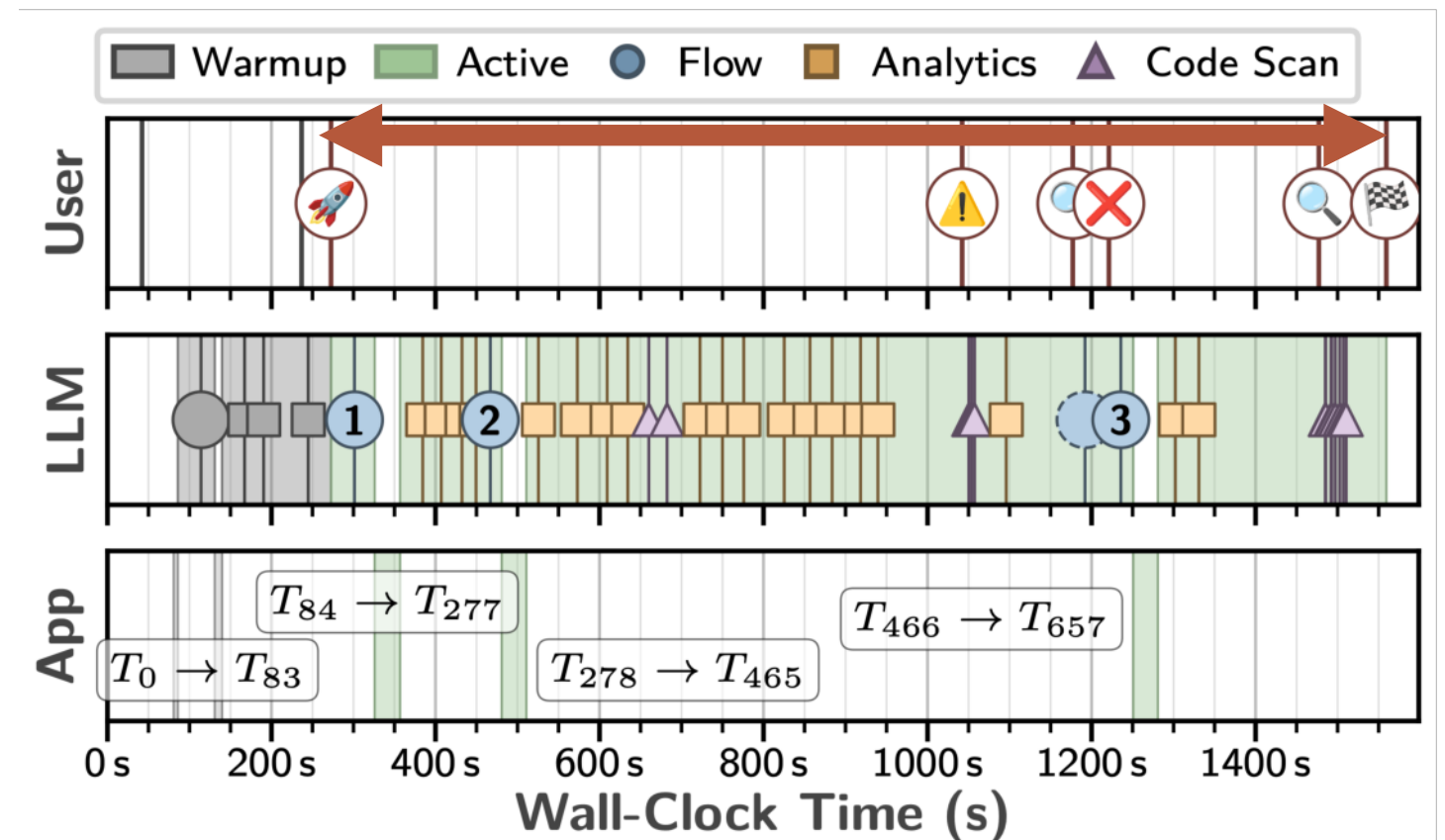
99.998% in-situ reduction @ 1.5% overhead
Lesson: use the supercomputer!

Agentic Eval: *LLM Debugs* in 27 mins!

- Goal: evaluate steerability using LLM (*Claude Opus 4.5*)
 - Asked to start with higher level metrics and localize
 - Reduced difficulty (*additional instrumentation*)
- Result: *done in 27 minutes*



Reduction in collected data

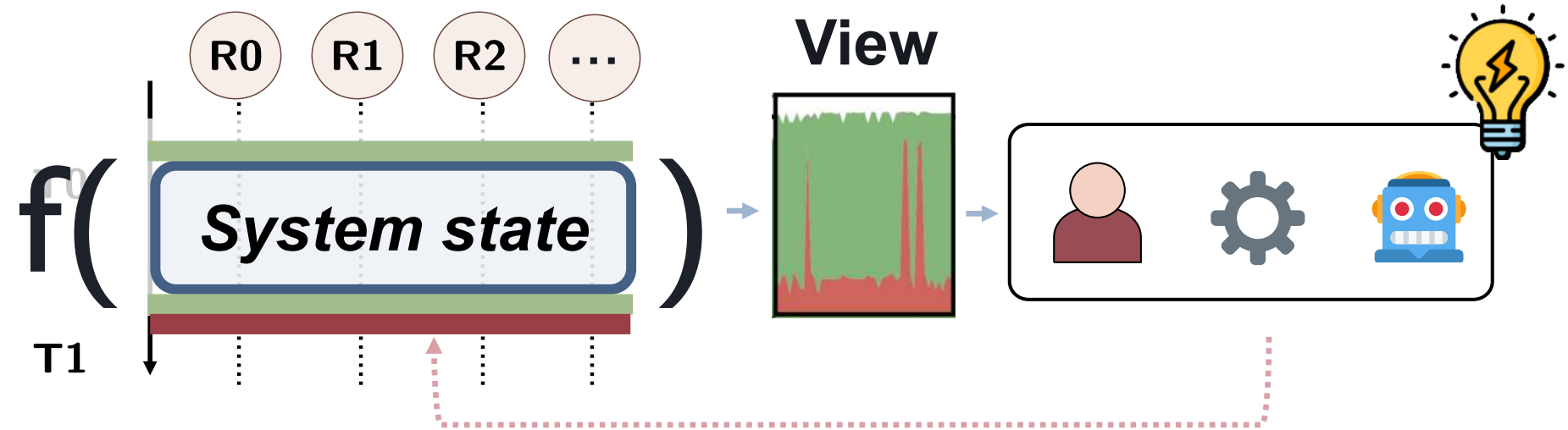
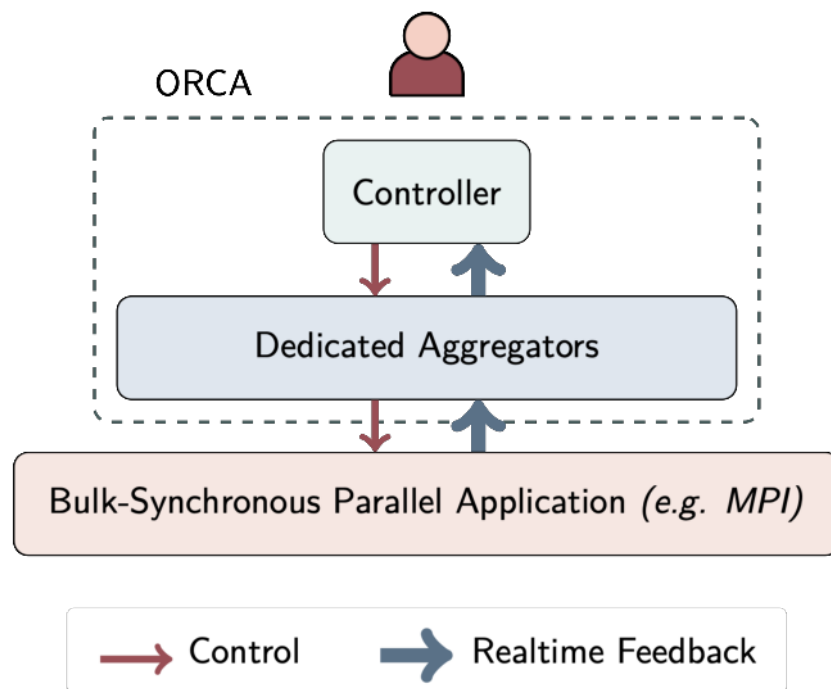


Talk Outline

- **Data path:** *CARP (Continuous Adaptive Range Partitioner)*
- **Compute path:** *Lessons from AMR Placement*
- **ORCA** (*Observability with Real-time Control and Aggregation*)
- **Conclusions**
 - **Future directions: *scientific analytics***
 - **Future directions: *ML observability***

Conclusion: *Unknowns Need Feedback Loops*

**System state has
~infinite resolution**



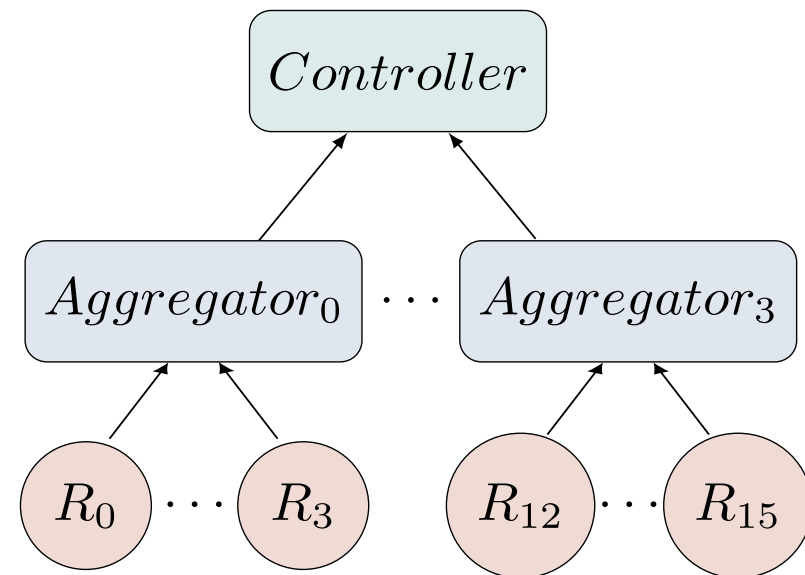
**Steerable Observability: Construct views instead
BSP: sync. defines *state* and *intervention***

ORCA: Always-on, BSP-aware, Steerable Observability

- *Spectrum of workflows: offline tracing to closed-loop*
- Leverages modern columnar analytics and RDMA
- *Lower overhead, smaller traces, faster analytics*

Future Directions: *Unified Scientific Analytics*

A Unified MapReduce Framework



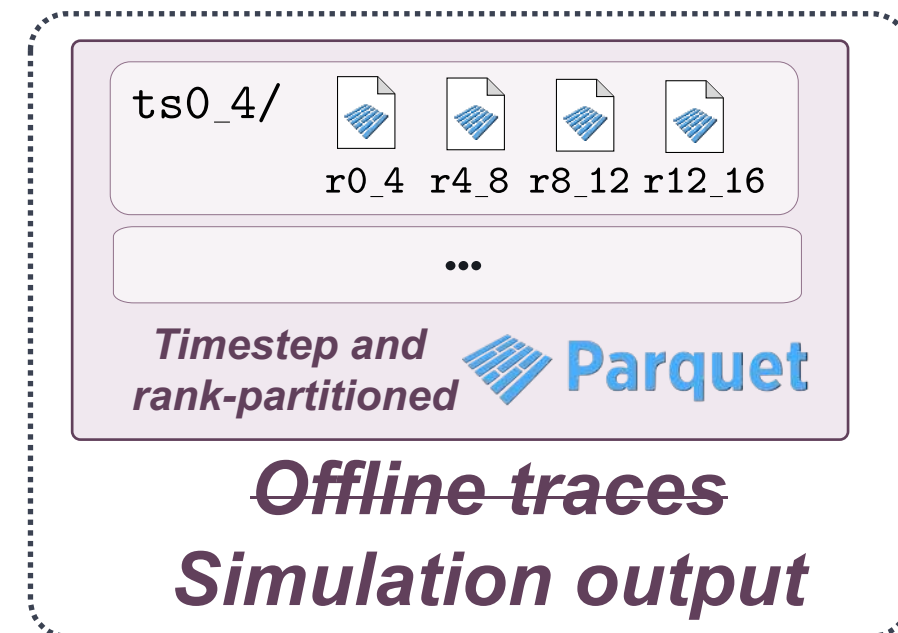
- **ORCA:** *Inter-tier* reductions
- **CARP:** *Intra-tier* shuffling

could unify in-situ visualization,
observability, indexing etc.*

Carnegie Mellon University

Parallel Data Laboratory

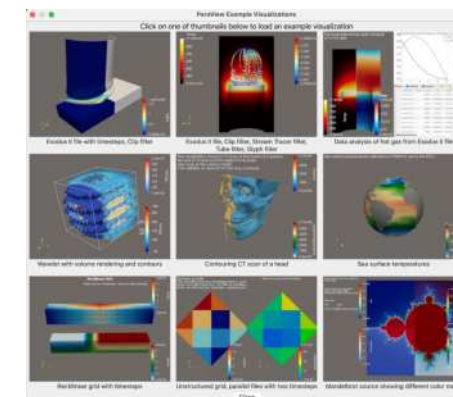
Common Analytics-friendly Formats



ICEBERG 

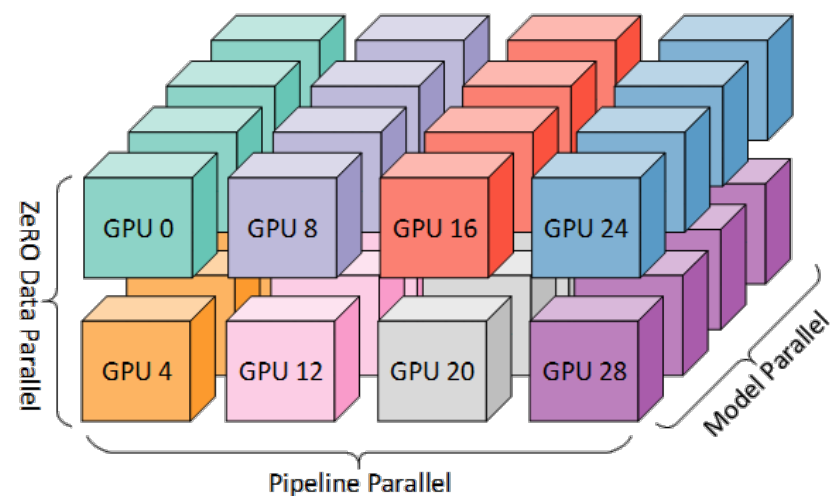


Ascent
ParaView

Future Directions: Observability for ML

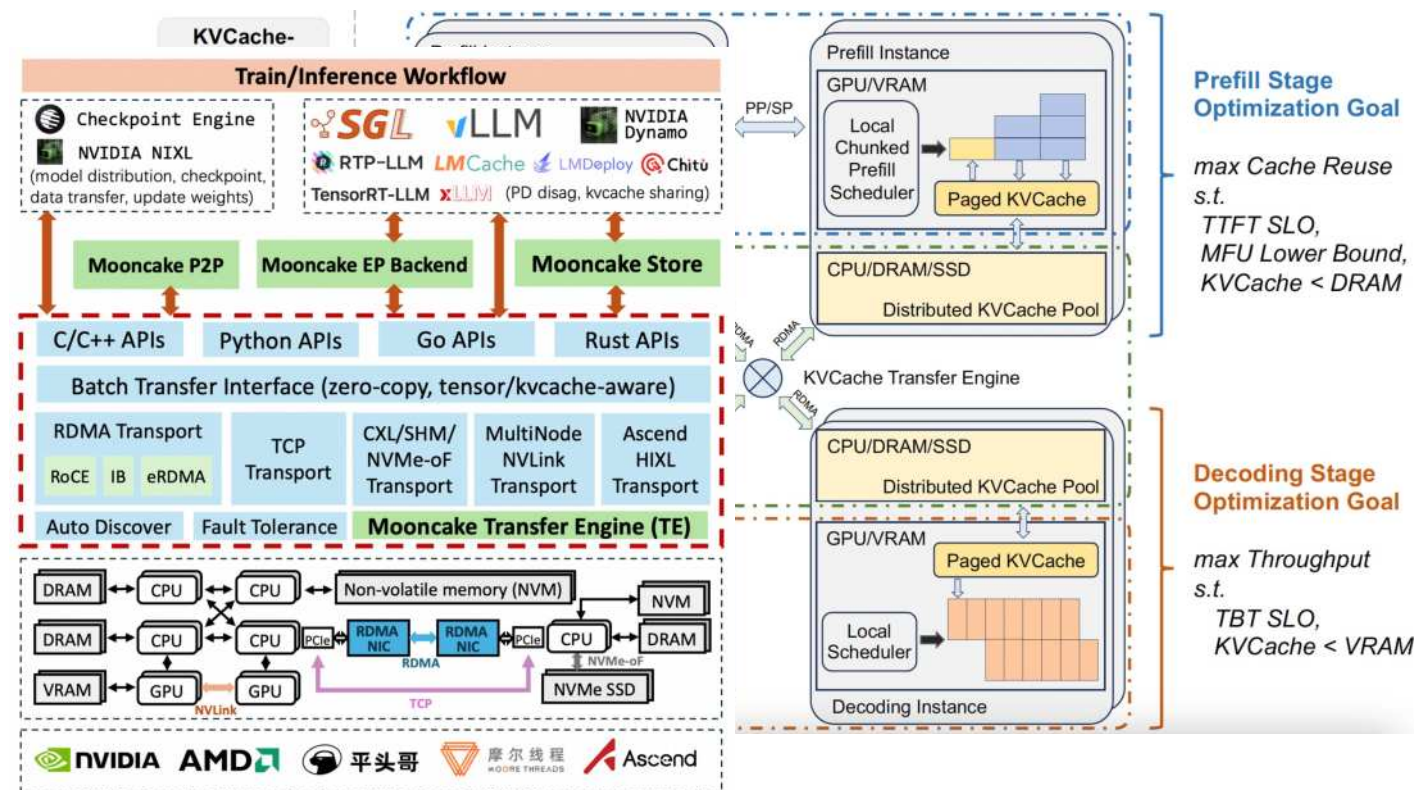
ML Training: *Still BSP,
but multidimensional*



Extensions

- Parallelism-aligned tree topology
- Parallelism-aligned pushdowns
- GPU-resident tensor analytics

ML Inference: *much smaller
BSP component (< 72 GPUs)*



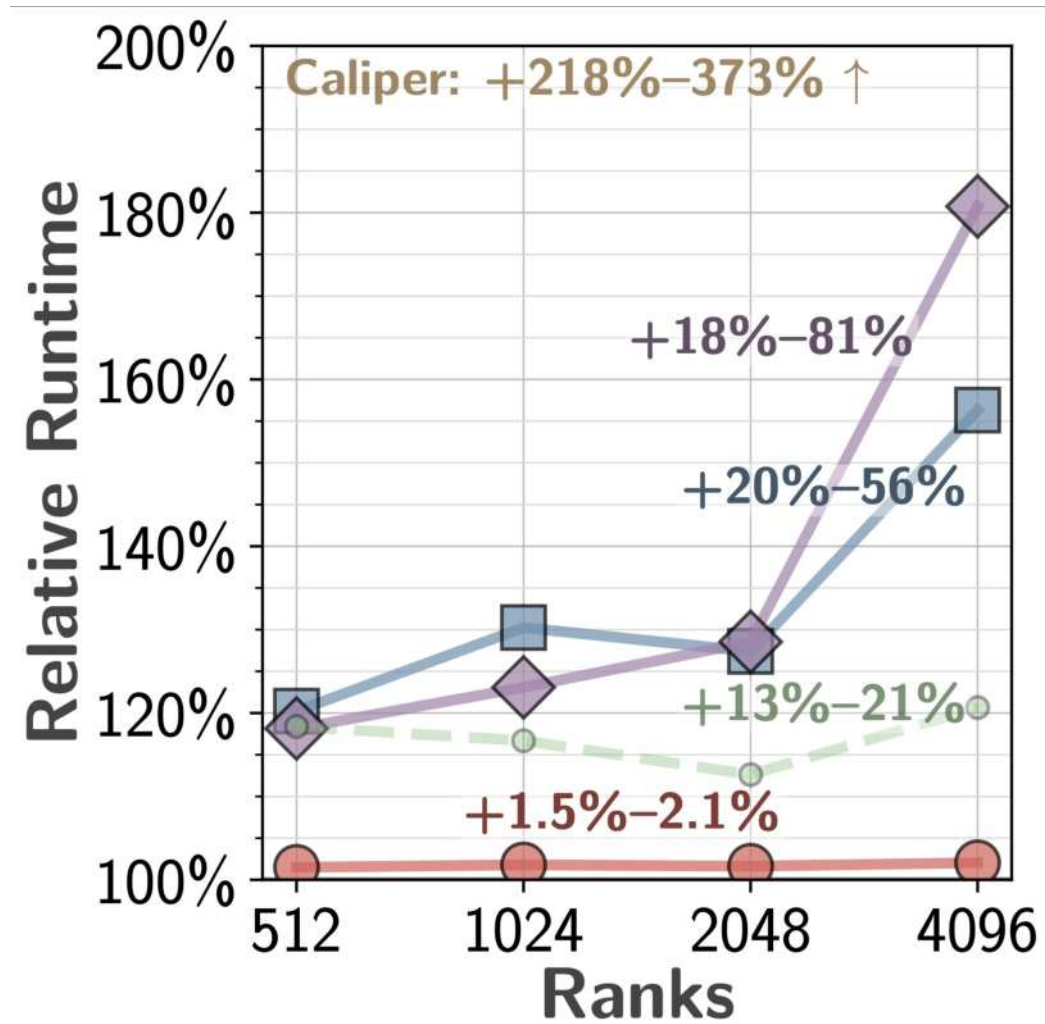
- Complex sync + async flows
- Collection/fidelity tradeoffs still apply
- ***Is Dapper still the gold standard?***

Backup Slides

Full Tracing: < 2% Runtime Overhead



**extrapolated from 200 timesteps*



Tracers

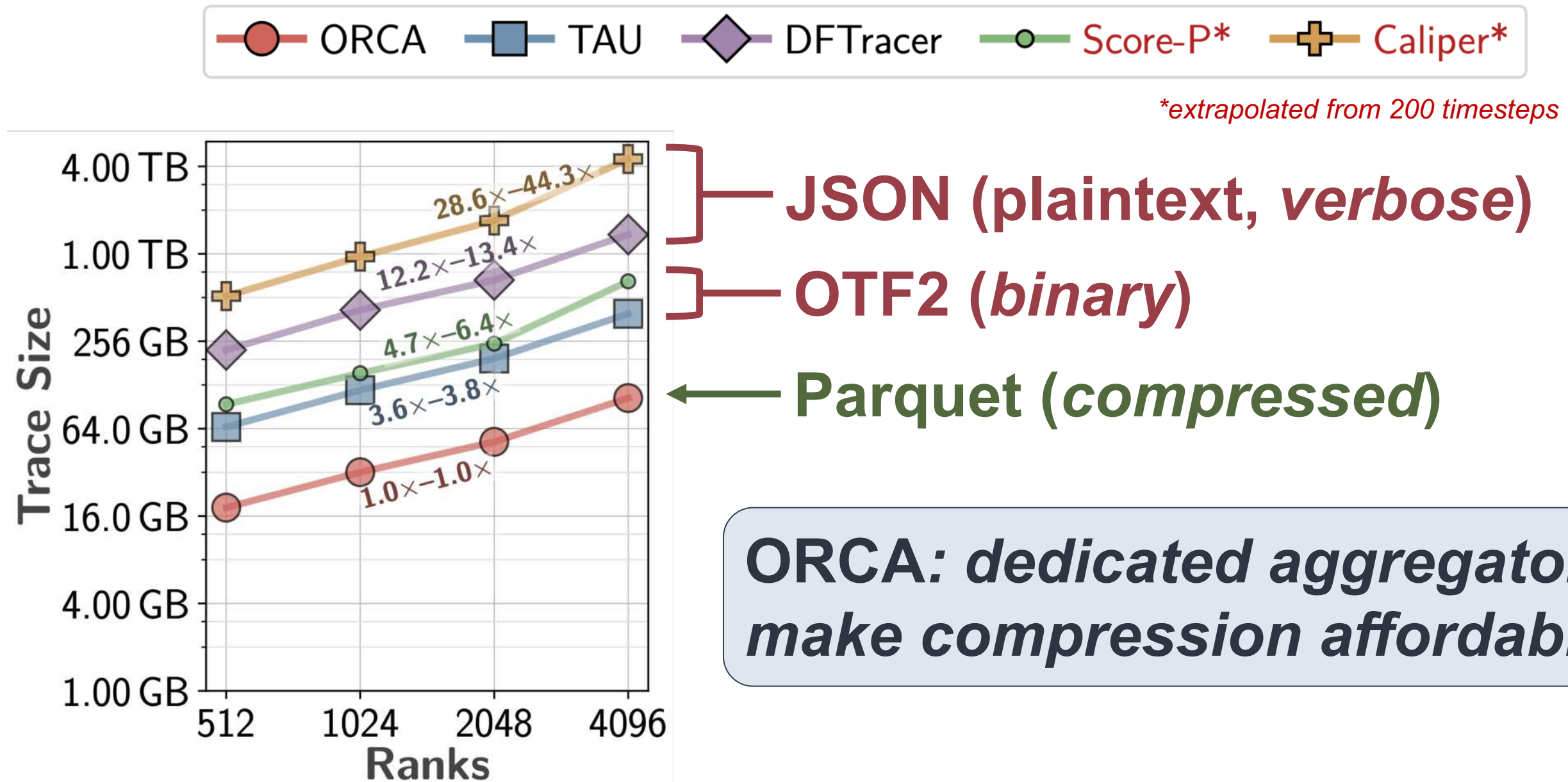
- I/O-driven jitter
- Storage bandwidth bottleneck

ORCA

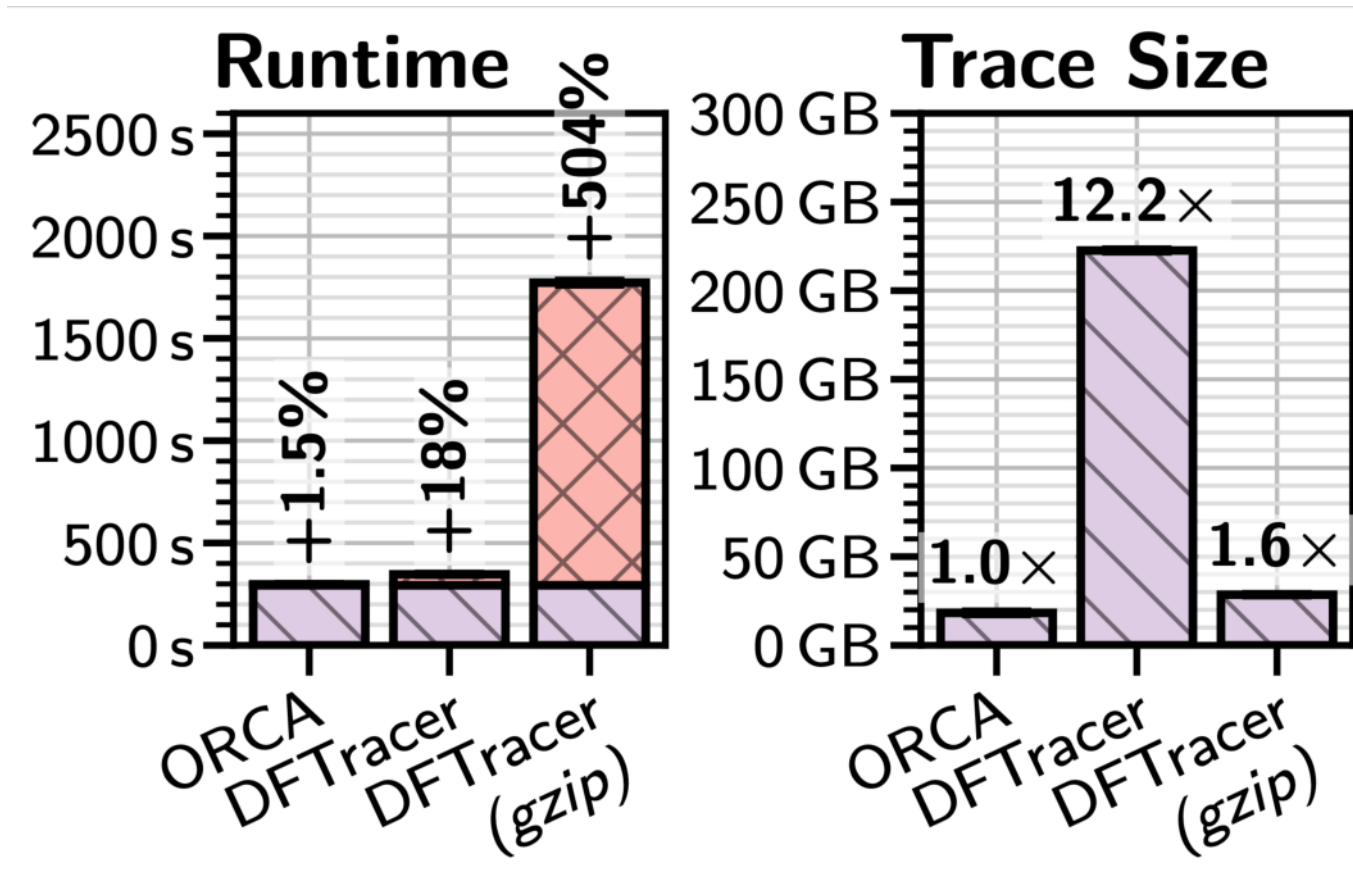
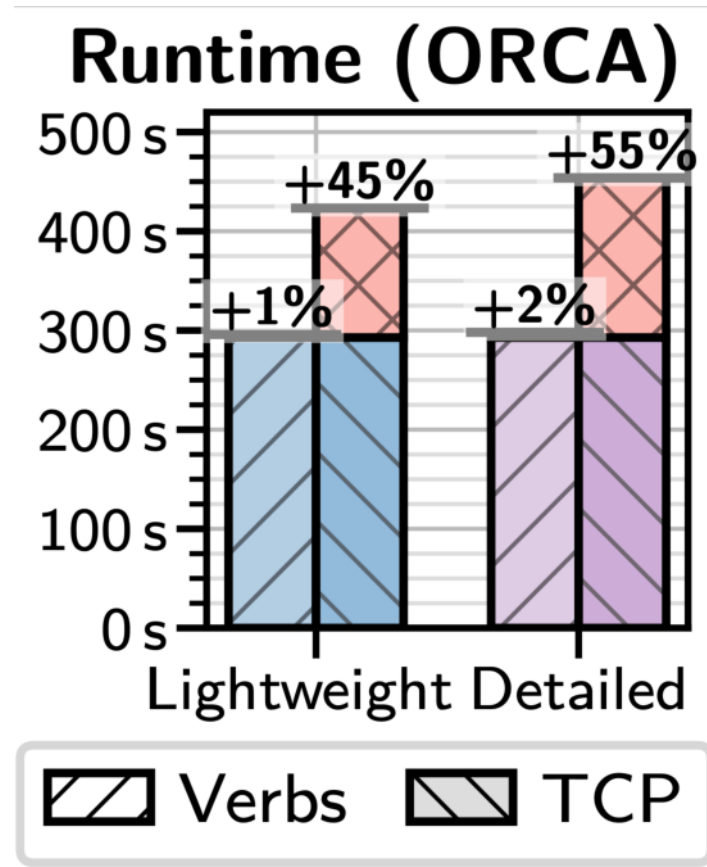
- RDMA, pull-based
- Compact traces

Lesson: use the fabric!

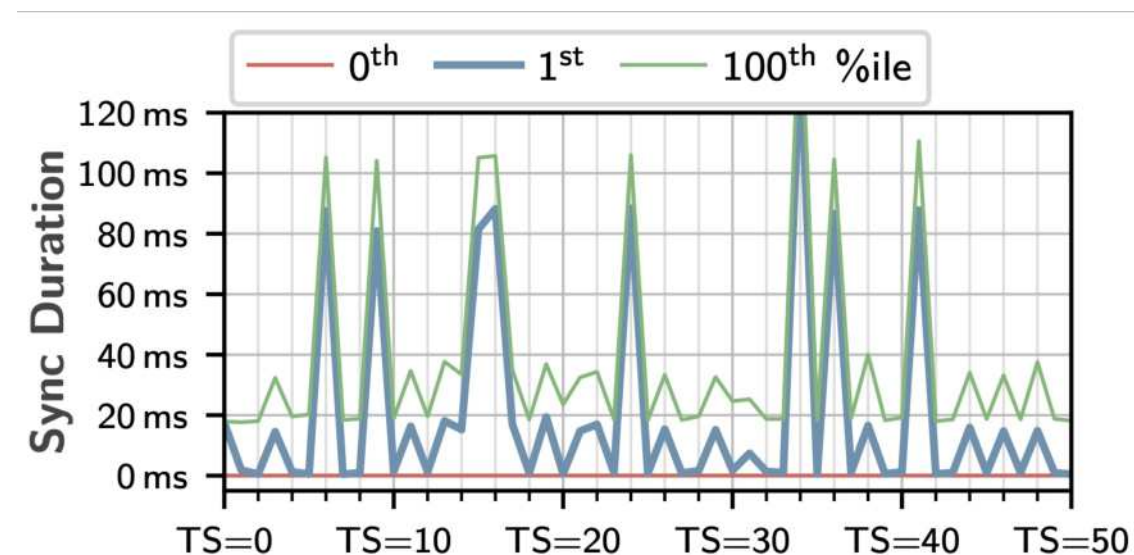
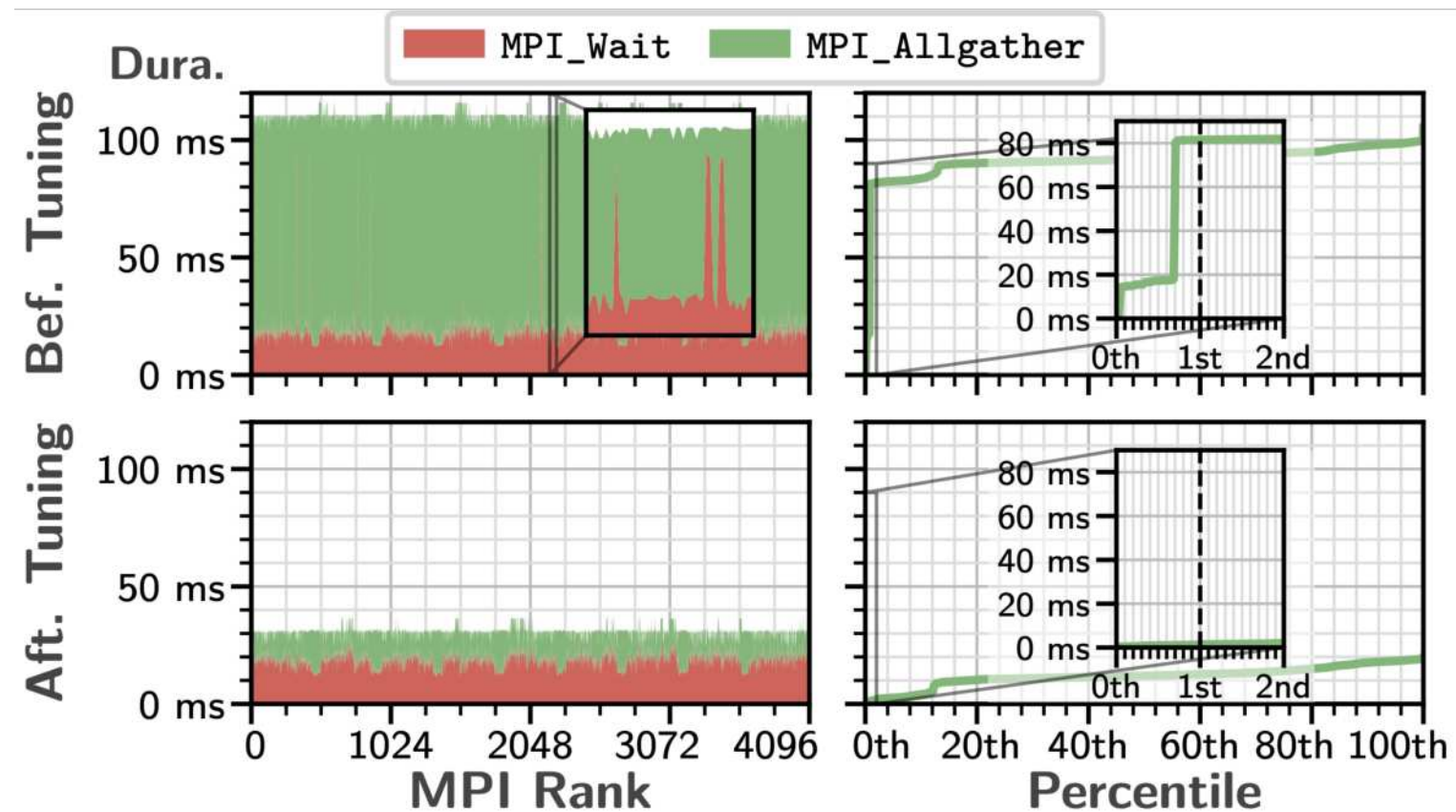
Full Tracing: Up to 44X Smaller Traces

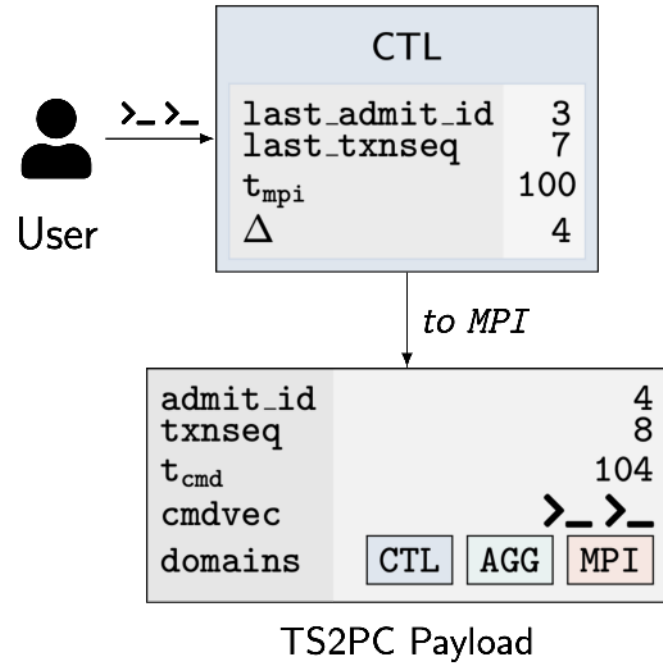


Architectural Analyses

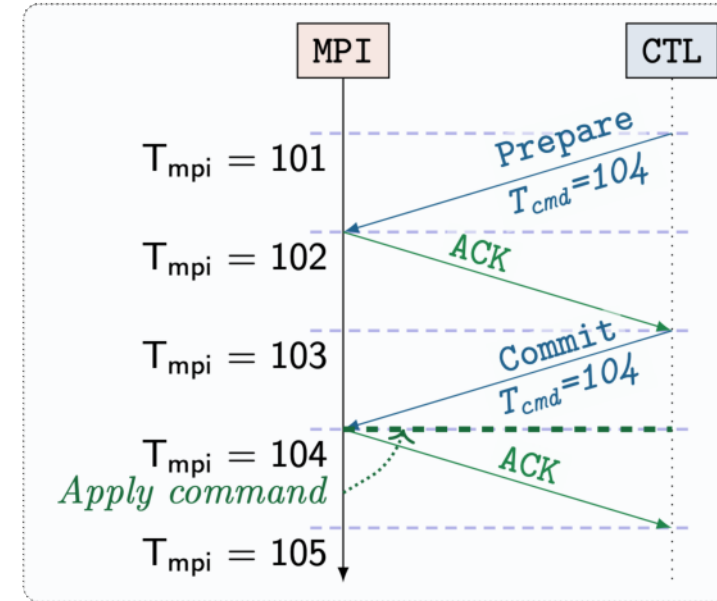


Volatile MPI_Wait: Different Views

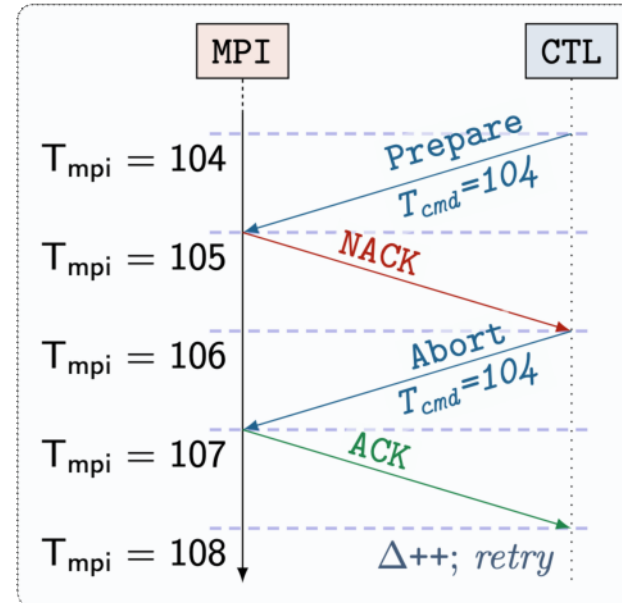




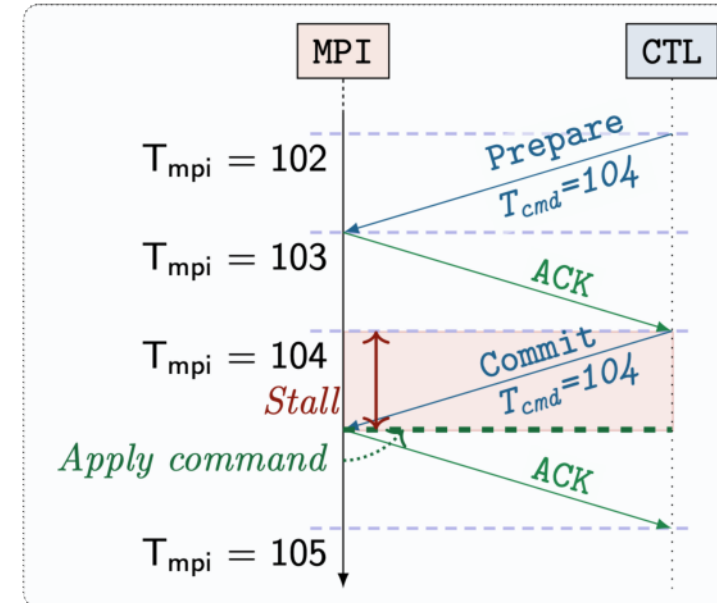
Case A. COMMIT (Fast)



Case B. ABORT



Case C. COMMIT (Slow)



----- Global Sync

Beyond Dapper?

- Ontologies
 - Dapper ~ only requests
 - Same event can be aggregated multiple ways
- Model-driven collection
 - Model ~ hypothesis
 - Persist only if violated
 - Persistence ~ core dump
- ORCA lessons:
 - Can do a lot more observability with Arrow + pushdown
 - Collect locally as ring buffers (maybe tiered)
 - Checkpoint ephemeral state if predicates violated
 - Seems to be a gap between heterogenous event tracing and distributed tracing

Slowdown event

- GROUPBY link
- GROUPBY node

Request slows down

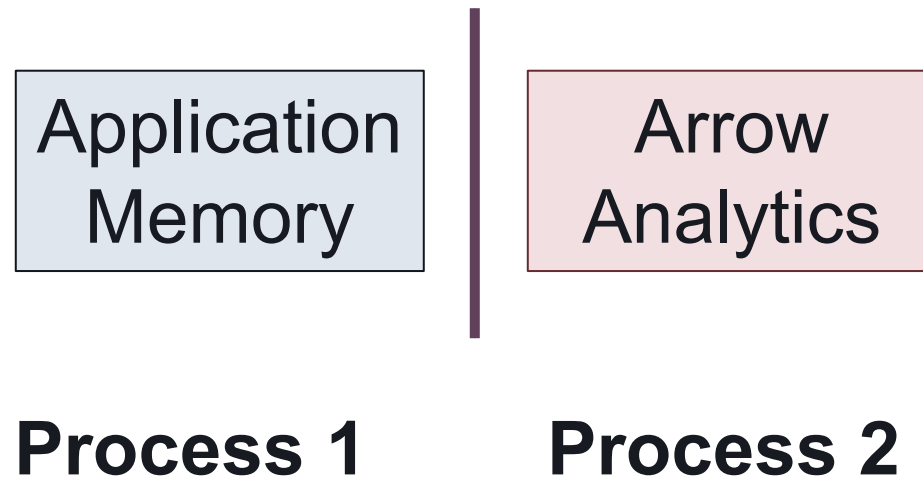
- *Also persist parent*
- *Need distributed predicates*

Beyond Dapper contd.

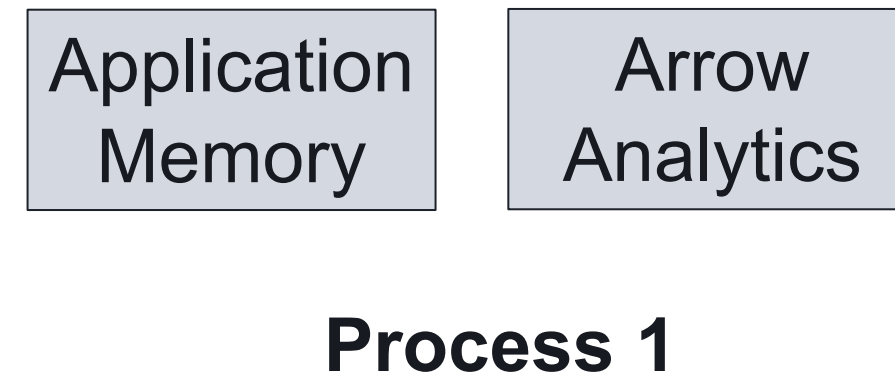
- Arrow allows composition of analytics approaches
 - Experimental pushdown + backend ingestion + ...
 - You can write to ClickHouse, or intercept and transform/filter/...
- Arrow enables an analytical definition of system state
 - DLPack etc.
 - Observability ~ analytics over distributed state

Definitions of System State

IPC (local)



Zero-copy



On Generality

System	Diagnostic Task	Their Approach	ORCA Equivalent
Greyhound [14] (Fail-slows)	Iteration boundary detection	Autocorrelation over collectives	PostTimestepAdvance
	Fail-slow detection	Track iteration times	Track iteration times
	Straggler localization	Aggregate NCCL events (off-fabric)	GROUPBY swid (on-fabric)
	Straggler cause disambiguation	Microbenchmarks	Microbenchmarks or trace events
	Online interventions	Tune micro-batch distribution or parallelism topology	Supported via control plane
Flare [72] (Training anomalies)	Stall detection/disambiguation	Call stack analysis	In-situ reductions
	Slowdown analysis	Periodic metrics	In-situ trace event aggregation
Holmes [15] (Fail-slows)	Outlier collective detection	Random forest model	Policy-dependent
	Localization	Graph search	Learn collective topology at bootstrap, GROUPBY swid, trace critical path in reverse
Mycroft [16] (Training anomalies)	Straggler detection/localization	Fine-grained trace NCCL events buffered locally, flushed to Kafka if heuristics trigger	Collective-aligned timestep dataframes, analyzed in-situ
TrainCheck [21] (Training correctness)	Correctness invariants	Offline trace analysis for invariant generation, online enforcement	Timestep-aligned traces for offline generation, in-situ enforcement via OrcaFlow
Minder [17] (Fault diagnosis)	Fail-slow hardware	Cluster-level metrics and anomaly detection	Workload-level trace events
Aegis [20] (Fault diagnosis)	Fail-stop hardware	Per-node NCCL counters pulled on timeouts	Workload-level trace events
Reveal [18] (Training anomalies)	Anomaly detection and bottleneck attribution	Time-windowed metrics + unsupervised learning	Workload-level trace events

Table 5.2: Diagnostic approaches in ML training observability and their ORCA equivalents.

Fay and Snicket

Fay, MSR SV, SOSP 2011

- LINQ/Dryand/FayLINQ
- Implementation closely coupled to Windows/.NET

Snicket, NYU/Princeton/Rutgers, HotNets '21

- Emits wasm filters

Pivot Tracing, Mace/Fonseca, SOSP 2015

- Causal graphs + happens-before

ORCA

- Same lineage, extended to BSP/RDMA
- + control guarantees
- + Arrow/Parquet/Datafusion: analytical flexibility/zero-copy etc.
- Composability with other in-situ analytical usecases
- ***Arrow/Datafusion make causal analyses practical?***